

```
%pip install -q "timm==1.0.19" "kagglehub==0.3.13" "scikit-learn==1.7.1" "pandas==2.2.3" "Pillow==11.3.0" "tqdm==4.67.1" "r
```

```
import copy
import hashlib
import json
import math
import platform
import random
import shutil
import time
from pathlib import Path

import kagglehub
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import sklearn
import timm
import torch
import torch.nn as nn
import torch.nn.functional as F
from matplotlib.colors import TwoSlopeNorm
from PIL import Image
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, roc_auc_score
from sklearn.model_selection import StratifiedGroupKFold
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms
from tqdm.auto import tqdm

SEED = 42
IMAGE_SIZE = 224
BATCH_SIZE = 64
NUM_WORKERS = 2
MAX_NEG_PER_PATIENT = 30
LABEL_FRACTIONS = [0.05, 0.10, 0.25, 1.00]
BUDGET_SEEDS = list(range(20))
REFERENCE_FULL_SEED = BUDGET_SEEDS[0]
C_GRID = [1e-3, 1e-2, 1e-1, 1.0, 10.0]
DEFAULT_C = 1.0

# Compute-conscious inference settings.
N_FULL_PATIENT_BOOTSTRAPS = 500
N_PRIMARY_BUDGET_BOOTSTRAPS = 2000
PRIMARY_BOOTSTRAP_SEED = SEED

# Safety switches.
ALLOW_DINO_TRAINING = False
ALLOW_FEATURE_EXTRACTION = False
ALLOW_VERIFIED_LEGACY_CACHE_MIGRATION = True

# Explainability uses the lower-label regime and a coarse first pass.
EXPLAIN_FRACTION = 0.25
EXPLAIN_SEED = BUDGET_SEEDS[0]
EXPLAIN_MODELS = ["vits_in", "dinov2_domain"]
RANDOM_CASES_PER_CLASS = 3
OCCLUSION_PATCH = 32
OCCLUSION_STRIDE = 32
OCCLUSION_BATCH = 16

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
torch.use_deterministic_algorithms(True, warn_only=True)
torch.backends.cudnn.benchmark = False

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
PIN_MEMORY = DEVICE.type == "cuda"

print("Python:", platform.python_version())
print("PyTorch:", torch.__version__)
print("torchvision:", __import__("torchvision").__version__)
print("timm:", timm.__version__)
print("scikit-learn:", sklearn.__version__)
print("pandas:", pd.__version__)
print("Device:", DEVICE)
if DEVICE.type == "cuda":
```

```

    print("GPU:", torch.cuda.get_device_name(0))
else:
    print("Warning: no GPU is active. Do not enable DINO training or feature extraction.")

```

```

Python: 3.13.15
PyTorch: 2.11.0+cu128
torchvision: 0.26.0+cu128
timm: 1.0.19
scikit-learn: 1.7.1
pandas: 2.2.3
Device: cuda
GPU: Tesla T4

```

```
from google.colab import drive
```

```
drive.mount("/content/drive")
```

```

PROJECT_ROOT = Path("/content/drive/MyDrive/isis_label_efficiency_v1")
FEATURE_DIR = PROJECT_ROOT / "features"
MODEL_DIR = PROJECT_ROOT / "models"
MANIFEST_DIR = PROJECT_ROOT / "manifests"

```

```

RUN_ID = "results_v10_final_a"
RESULT_DIR = PROJECT_ROOT / RUN_ID

```

```

for directory in [FEATURE_DIR, MODEL_DIR, MANIFEST_DIR, RESULT_DIR]:
    directory.mkdir(parents=True, exist_ok=True)

```

```

existing_outputs = sorted(path.name for path in RESULT_DIR.iterdir())
print("Project directory:", PROJECT_ROOT)
print("V10 result directory:", RESULT_DIR)
print("Existing V10 outputs:", len(existing_outputs))
if existing_outputs:
    print("Resuming this V10 run. Existing files will be overwritten by their cells.")

```

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
Project directory: /content/drive/MyDrive/isis_label_efficiency_v1
V10 result directory: /content/drive/MyDrive/isis_label_efficiency_v1/results_v10_final_a
Existing V10 outputs: 16
Resuming this V10 run. Existing files will be overwritten by their cells.

```

```

DATASET_HANDLE = "roscoekerby/isis-2024-permissive-training-input"
DATA_ROOT = Path(kagglehub.dataset_download(DATASET_HANDLE))
print("Dataset root:", DATA_ROOT)

```

```

csv_files = list(DATA_ROOT.rglob("*.csv"))
metadata_matches = [p for p in csv_files if p.name.lower() == "metadata.csv"]
ground_truth_matches = [p for p in csv_files if "groundtruth" in p.name.lower()]
assert len(metadata_matches) == 1, f"Expected one metadata.csv, found: {metadata_matches}"
assert len(ground_truth_matches) == 1, f"Expected one ground-truth CSV, found: {ground_truth_matches}"

```

```

metadata_df = pd.read_csv(metadata_matches[0])
ground_truth_df = pd.read_csv(ground_truth_matches[0])
assert {"isic_id", "patient_id"}.issubset(metadata_df.columns)
assert {"isic_id", "malignant"}.issubset(ground_truth_df.columns)
assert metadata_df["isic_id"].is_unique, "metadata.csv has duplicate isic_id values"
assert ground_truth_df["isic_id"].is_unique, "Ground truth has duplicate isic_id values"

```

```

metadata_df["isic_id"] = metadata_df["isic_id"].astype(str)
ground_truth_df["isic_id"] = ground_truth_df["isic_id"].astype(str)
ground_truth_df["target"] = pd.to_numeric(ground_truth_df["malignant"], errors="coerce")

```

```

df = metadata_df.merge(
    ground_truth_df[["isic_id", "target"]],
    on="isic_id",
    how="inner",
    validate="one_to_one",
)
df = df.dropna(subset=["patient_id", "target"]).copy()
df["patient_id"] = df["patient_id"].astype(str)
df["target"] = df["target"].astype(int)
assert set(df["target"].unique()).issubset({0, 1})

```

```

image_files = list(DATA_ROOT.rglob("*.jpg")) + list(DATA_ROOT.rglob("*.jpeg"))
assert image_files, "No JPG/JPEG files were found"
image_table = pd.DataFrame({
    "isic_id": [p.stem for p in image_files],
    "image_path": [str(p) for p in image_files],
})
duplicate_stems = image_table.loc[image_table["isic_id"].duplicated(False), "isic_id"].unique()
assert len(duplicate_stems) == 0, f"Duplicate image stems found, for example: {duplicate_stems[:10]}"

```

```

before_images = len(df)
df = df.merge(image_table, on="isic_id", how="inner", validate="one_to_one")
df = df.reset_index(drop=True)
assert len(df) > 0
for path in df["image_path"].sample(min(20, len(df)), random_state=SEED):
    assert Path(path).is_file(), path

print(f"Usable images: {len(df):,} of {before_images:,} labelled rows")
print(df["target"].value_counts().sort_index())
print(f"Unique patients: {df['patient_id'].nunique():,}")

```

Using Colab cache for faster access to the 'isic-2024-permissive-training-input' dataset.  
Dataset root: /kaggle/input/isic-2024-permissive-training-input  
Usable images: 217,477 of 217,477 labelled rows

```

target
0    217183
1         294
Name: count, dtype: int64
Unique patients: 634

```

```

SPLIT_MANIFEST_PATH = MANIFEST_DIR / f"patient_split_seed_{SEED}.csv"
if not SPLIT_MANIFEST_PATH.is_file():
    raise FileNotFoundError(
        f"Missing frozen split manifest: {SPLIT_MANIFEST_PATH}. "
        "Do not create a new split on deadline day; recover the V9 manifest."
    )

# Load identifiers and split membership only. The locked test frame is never built.
split_manifest = pd.read_csv(
    SPLIT_MANIFEST_PATH,
    usecols=["isic_id", "patient_id", "split"],
    dtype={"isic_id": str, "patient_id": str, "split": str},
)
assert split_manifest["isic_id"].is_unique
assert set(split_manifest["split"]) == {"train", "validation", "test"}

split_lookup = split_manifest[["isic_id", "split"]]
indexed_df = df.merge(split_lookup, on="isic_id", how="inner", validate="one_to_one")
assert len(indexed_df) == len(split_manifest), "Dataset and frozen manifest differ"

train_pool = (
    indexed_df[indexed_df["split"] == "train"]
    .drop(columns="split")
    .reset_index(drop=True)
)
val_df = (
    indexed_df[indexed_df["split"] == "validation"]
    .drop(columns="split")
    .reset_index(drop=True)
)
locked_test_patient_ids = set(
    split_manifest.loc[split_manifest["split"] == "test", "patient_id"]
)

def show_split(name, frame):
    positive_patients = frame.loc[frame["target"] == 1, "patient_id"].nunique()
    print(
        f"{name:12} | images={len(frame):,} | patients={frame['patient_id'].nunique():,} | "
        f"positive images={int(frame['target'].sum()):,} | positive patients={positive_patients:,} | "
        f"prevalence={frame['target'].mean():.4%}"
    )

for name, frame in [("Train", train_pool), ("Validation", val_df)]:
    assert frame["target"].nunique() == 2, f"{name} has only one class"
    show_split(name, frame)

train_patients = set(train_pool["patient_id"])
val_patients = set(val_df["patient_id"])
assert train_patients.isdisjoint(val_patients)
assert train_patients.isdisjoint(locked_test_patient_ids)
assert val_patients.isdisjoint(locked_test_patient_ids)
print("Locked test patients (not loaded or scored):", len(locked_test_patient_ids))

Train      | images=134,874 | patients=381 | positive images=185 | positive patients=101 | prevalence=0.1372%
Validation  | images=36,070 | patients=126 | positive images=62 | positive patients=37 | prevalence=0.1719%
Locked test patients (not loaded or scored): 127

```

# step 6- Define patient budget sampling and the probe training cap

```

def patient_specific_seed(patient_id, base_seed):
    digest = hashlib.sha256(f"{base_seed}|{patient_id}".encode()).digest()
    return int.from_bytes(digest[:8], "little", signed=False)

```

```
def cap_for_probe(frame, max_negatives, seed=SEED):
    """Keep every positive and at most max_negatives negatives per patient.

    This is a post-annotation training/compute operation. It must never be
    presented as reducing the number of annotations acquired.
    """
    kept = []
    for patient_id, group in frame.groupby("patient_id", sort=True):
        positives = group[group["target"] == 1]
        negatives = group[group["target"] == 0]
        if len(negatives) > max_negatives:
            rng = np.random.default_rng(patient_specific_seed(patient_id, seed))
            take = rng.choice(len(negatives), size=max_negatives, replace=False)
            negatives = negatives.iloc[take]
        kept.append(pd.concat([positives, negatives]))
    return pd.concat(kept).sort_values("isic_id").reset_index(drop=True)

def select_patient_budget(frame, fraction, seed):
    """Select patients without consulting their labels."""
    rng = np.random.default_rng(seed)
    patients = frame["patient_id"].drop_duplicates().to_numpy()
    n_patients = max(2, int(round(len(patients) * fraction)))
    chosen = np.sort(rng.choice(patients, size=n_patients, replace=False))

    # Labels are used below only after the simulated annotation selection.
    labelled = frame[frame["patient_id"].isin(chosen)]
    report = {
        "n_labelled_patients": int(labelled["patient_id"].nunique()),
        "n_labelled_images": int(len(labelled)),
        "n_labelled_positive_images": int(labelled["target"].sum()),
        "n_labelled_positive_patients": int(
            labelled.loc[labelled["target"] == 1, "patient_id"].nunique()
        ),
    }
    return chosen, report

probe_pool = cap_for_probe(train_pool, MAX_NEG_PER_PATIENT)
show_split("Probe pool", probe_pool)
assert probe_pool["target"].sum() == train_pool["target"].sum()
assert set(probe_pool["patient_id"]) == set(train_pool["patient_id"])
```

Probe pool | images=11,231 | patients=381 | positive images=185 | positive patients=101 | prevalence=1.6472%

# step 7 - Define deterministic image pre-processing

```
eval_transform = transforms.Compose([
    transforms.Resize(256, interpolation=transforms.InterpolationMode.BICUBIC),
    transforms.CenterCrop(IMAGE_SIZE),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225],
    ),
])

class LesionDataset(Dataset):
    def __init__(self, frame, transform):
        self.frame = frame.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.frame)

    def __getitem__(self, index):
        row = self.frame.iloc[index]
        with Image.open(row.image_path) as image:
            tensor = self.transform(image.convert("RGB"))
        return tensor, torch.tensor(row.target, dtype=torch.float32)
```

# step 8 - define domain adaption configuration

```
DINO_DOMAIN_MODEL_KEY = "dinov2_domain"
DINO_BASE_MODEL_NAME = "vit_small_patch14_reg4_dinov2.lvd142m"
DINO_ADAPT_EPOCHS = 2
DINO_ADAPT_BATCH_SIZE = 32
DINO_ADAPT_MAX_IMAGES = 12_000
DINO_ADAPT_MAX_IMAGES_PER_PATIENT = 30
DINO_ADAPT_UNFREEZE_BLOCKS = 2
DINO_ADAPT_LEARNING_RATE = 2e-5
```

```

DINO_ADAPT_HEAD_LEARNING_RATE = 1e-3
DINO_ADAPT_WEIGHT_DECAY = 0.04
DINO_ADAPT_TEACHER_MOMENTUM = 0.996
DINO_STUDENT_TEMPERATURE = 0.10
DINO_TEACHER_TEMPERATURE = 0.04
DINO_CENTER_MOMENTUM = 0.90
DINO_PROJECTION_HIDDEN_DIM = 1024
DINO_PROJECTION_BOTTLENECK_DIM = 256
DINO_PROJECTION_OUTPUT_DIM = 2048
FORCE_DINO_DOMAIN_ADAPTATION = False

MODEL_DIR = PROJECT_ROOT / "models"
MODEL_DIR.mkdir(parents=True, exist_ok=True)
DINO_DOMAIN_CHECKPOINT = (
    MODEL_DIR / "dinov2_isic_domain_adapted_v2.pt"
)

def make_unlabelled_adaptation_pool(
    frame,
    max_images=DINO_ADAPT_MAX_IMAGES,
    max_images_per_patient=DINO_ADAPT_MAX_IMAGES_PER_PATIENT,
    seed=SEED,
):
    """Sample images using IDs only; target labels are deliberately ignored."""
    rng = np.random.default_rng(seed)
    kept = []
    safe_columns = ["isic_id", "patient_id", "image_path"]
    for _, group in frame[safe_columns].groupby("patient_id", sort=True):
        group = group.sort_values("isic_id")
        if len(group) > max_images_per_patient:
            positions = np.sort(
                rng.choice(len(group), size=max_images_per_patient, replace=False)
            )
            group = group.iloc[positions]
            kept.append(group)
    pool = pd.concat(kept, ignore_index=True)
    if len(pool) > max_images:
        positions = np.sort(rng.choice(len(pool), size=max_images, replace=False))
        pool = pool.iloc[positions]
    return pool.sort_values("isic_id").reset_index(drop=True)

dino_adaptation_pool = make_unlabelled_adaptation_pool(train_pool)
assert set(dino_adaptation_pool["patient_id"]).issubset(set(train_pool["patient_id"]))
assert set(dino_adaptation_pool["patient_id"]).isdisjoint(set(val_df["patient_id"]))
assert set(dino_adaptation_pool["patient_id"]).isdisjoint(locked_test_patient_ids)
print(
    f"DINO domain-adaptation pool: {len(dino_adaptation_pool):,} images from "
    f"{dino_adaptation_pool['patient_id'].nunique():,} training patients"
)

dino_view_transform = transforms.Compose([
    transforms.RandomResizedCrop(
        IMAGE_SIZE,
        scale=(0.50, 1.00),
        interpolation=transforms.InterpolationMode.BICUBIC,
    ),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomApply(
        [transforms.ColorJitter(brightness=0.30, contrast=0.30, saturation=0.20, hue=0.05)],
        p=0.80,
    ),
    transforms.RandomGrayscale(p=0.10),
    transforms.RandomApply([transforms.GaussianBlur(kernel_size=9)], p=0.30),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225],
    ),
])

def dino_adaptation_signature(frame):
    frame_hash = pd.util.hash_pandas_object(
        frame[["isic_id", "patient_id"]], index=False
    ).values.tobytes()
    config = {
        "base_model": DINO_BASE_MODEL_NAME,
        "epochs": DINO_ADAPT_EPOCHS,
        "batch_size": DINO_ADAPT_BATCH_SIZE,
        "max_images": DINO_ADAPT_MAX_IMAGES,
    }

```

```

        "max_images_per_patient": DINO_ADAPT_MAX_IMAGES_PER_PATIENT,
        "unfreeze_blocks": DINO_ADAPT_UNFREEZE_BLOCKS,
        "learning_rate": DINO_ADAPT_LEARNING_RATE,
        "head_learning_rate": DINO_ADAPT_HEAD_LEARNING_RATE,
        "weight_decay": DINO_ADAPT_WEIGHT_DECAY,
        "teacher_momentum": DINO_ADAPT_TEACHER_MOMENTUM,
        "student_temperature": DINO_STUDENT_TEMPERATURE,
        "teacher_temperature": DINO_TEACHER_TEMPERATURE,
        "center_momentum": DINO_CENTER_MOMENTUM,
        "projection_dims": [
            DINO_PROJECTION_HIDDEN_DIM,
            DINO_PROJECTION_BOTTLENECK_DIM,
            DINO_PROJECTION_OUTPUT_DIM,
        ],
        "seed": SEED,
        "image_size": IMAGE_SIZE,
        "timm": timm.__version__,
        "torch": str(torch.__version__),
    }
    digest = hashlib.sha256(
        frame_hash + json.dumps(config, sort_keys=True).encode()
    ).hexdigest()[:20]
    return digest, config

DINO_DOMAIN_SIGNATURE, DINO_DOMAIN_CONFIG = dino_adaptation_signature(
    dino_adaptation_pool
)

```

DINO domain-adaptation pool: 11,046 images from 381 training patients

```

def file_sha256(path, chunk_size=1024 * 1024):
    digest = hashlib.sha256()
    with Path(path).open("rb") as handle:
        while chunk := handle.read(chunk_size):
            digest.update(chunk)
    return digest.hexdigest()

```

# step 9 - load the domain checkpoint,the compute its Secure Hash Algorithm (SHA)

```

class DINOViewDataset(Dataset):
    def __init__(self, frame, transform):
        self.frame = frame.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.frame)

    def __getitem__(self, index):
        image_path = self.frame.iloc[index]["image_path"]
        with Image.open(image_path) as image:
            image = image.convert("RGB")
            first_view = self.transform(image)
            second_view = self.transform(image)
        return first_view, second_view

class DINOProjectionHead(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.mlp = nn.Sequential(
            nn.Linear(input_dim, DINO_PROJECTION_HIDDEN_DIM),
            nn.GELU(),
            nn.Linear(DINO_PROJECTION_HIDDEN_DIM, DINO_PROJECTION_BOTTLENECK_DIM),
        )
        self.prototypes = nn.Linear(
            DINO_PROJECTION_BOTTLENECK_DIM,
            DINO_PROJECTION_OUTPUT_DIM,
            bias=False,
        )

    def forward(self, features):
        projected = F.normalize(self.mlp(features), dim=-1, eps=1e-6)
        return self.prototypes(projected)

def set_dino_adaptation_trainable(backbone):
    for parameter in backbone.parameters():
        parameter.requires_grad_(False)
    for block in backbone.blocks[-DINO_ADAPT_UNFREEZE_BLOCKS:]:

```

```

        for parameter in block.parameters():
            parameter.requires_grad_(True)
    for parameter in backbone.norm.parameters():
        parameter.requires_grad_(True)

def dino_cross_view_loss(student_logits, teacher_logits, center):
    student_first, student_second = student_logits.float().chunk(2)
    teacher_first, teacher_second = teacher_logits.float().chunk(2)
    teacher_first = torch.softmax(
        (teacher_first - center) / DINO_TEACHER_TEMPERATURE,
        dim=-1,
    ).detach()
    teacher_second = torch.softmax(
        (teacher_second - center) / DINO_TEACHER_TEMPERATURE,
        dim=-1,
    ).detach()
    student_first = torch.log_softmax(
        student_first / DINO_STUDENT_TEMPERATURE,
        dim=-1,
    )
    student_second = torch.log_softmax(
        student_second / DINO_STUDENT_TEMPERATURE,
        dim=-1,
    )
    first_to_second = -(teacher_first * student_second).sum(dim=-1).mean()
    second_to_first = -(teacher_second * student_first).sum(dim=-1).mean()
    return 0.5 * (first_to_second + second_to_first)

@torch.no_grad()
def ema_update(teacher_module, student_module, momentum):
    for teacher_parameter, student_parameter in zip(
        teacher_module.parameters(), student_module.parameters()
    ):
        teacher_parameter.mul_(momentum).add_(
            student_parameter.detach(), alpha=1.0 - momentum
        )

def load_checkpoint_safely(path):
    from torch.torch_version import TorchVersion

    with torch.serialization.safe_globals([TorchVersion]):
        return torch.load(
            path,
            map_location="cpu",
            weights_only=True,
        )

```

```

def train_or_load_domain_dino():
    if DINO_DOMAIN_CHECKPOINT.is_file() and not FORCE_DINO_DOMAIN_ADAPTATION:
        checkpoint = load_checkpoint_safely(DINO_DOMAIN_CHECKPOINT)
        saved_signature = checkpoint.get("metadata", {}).get("signature")
        if saved_signature == DINO_DOMAIN_SIGNATURE:
            print("Using matching domain-adapted DINO checkpoint:", DINO_DOMAIN_CHECKPOINT)
            return checkpoint
        print("Existing DINO checkpoint has a stale signature; retraining it.")

    if not ALLOW_DINO_TRAINING:
        raise RuntimeError(
            "Matching DINO checkpoint not found. Leave this notebook stopped "
            "unless you intentionally set ALLOW_DINO_TRAINING=True."
        )

    if DEVICE.type != "cuda":
        print("Warning: DINO domain adaptation is practical on a GPU but will run on CPU.")

    torch.use_deterministic_algorithms(True, warn_only=False)
    if DEVICE.type == "cuda":
        torch.backends.cuda.enable_flash_sdp(False)
        torch.backends.cuda.enable_mem_efficient_sdp(False)
        torch.backends.cuda.enable_math_sdp(True)

    student_backbone = timm.create_model(
        DINO_BASE_MODEL_NAME,
        pretrained=True,
        num_classes=0,
        img_size=IMAGE_SIZE,
    ).to(DEVICE)
    teacher_backbone = copy.deepcopy(student_backbone).to(DEVICE)
    set_dino_adaptation_trainable(student_backbone)

```

```

student_backbone.train()
teacher_backbone.eval()
for parameter in teacher_backbone.parameters():
    parameter.requires_grad_(False)

feature_dim = int(student_backbone.num_features)
student_head = DINOProjectionHead(feature_dim).to(DEVICE)
teacher_head = copy.deepcopy(student_head).to(DEVICE).eval()
for parameter in teacher_head.parameters():
    parameter.requires_grad_(False)

trainable_backbone_parameters = [
    parameter for parameter in student_backbone.parameters()
    if parameter.requires_grad
]
optimizer = torch.optim.AdamW(
    [
        {
            "params": trainable_backbone_parameters,
            "lr": DINO_ADAPT_LEARNING_RATE,
            "base_lr": DINO_ADAPT_LEARNING_RATE,
        },
        {
            "params": student_head.parameters(),
            "lr": DINO_ADAPT_HEAD_LEARNING_RATE,
            "base_lr": DINO_ADAPT_HEAD_LEARNING_RATE,
        },
    ],
    weight_decay=DINO_ADAPT_WEIGHT_DECAY,
)
scaler = torch.amp.GradScaler(
    "cuda",
    enabled=DEVICE.type == "cuda",
)

def seed_worker(worker_id):
    worker_seed = torch.initial_seed() % (2**32)
    np.random.seed(worker_seed)
    random.seed(worker_seed)

generator = torch.Generator()
generator.manual_seed(SEED)
loader = DataLoader(
    DINOViewDataset(dino_adaptation_pool, dino_view_transform),
    batch_size=DINO_ADAPT_BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS,
    pin_memory=PIN_MEMORY,
    persistent_workers=NUM_WORKERS > 0,
    drop_last=len(dino_adaptation_pool) >= DINO_ADAPT_BATCH_SIZE,
    worker_init_fn=seed_worker,
    generator=generator,
)
total_steps = max(1, DINO_ADAPT_EPOCHS * len(loader))
center = torch.zeros(1, DINO_PROJECTION_OUTPUT_DIM, device=DEVICE)
history = []
global_step = 0

for epoch in range(DINO_ADAPT_EPOCHS):
    running_loss = 0.0
    progress = tqdm(loader, desc=f"DINO domain adaptation {epoch + 1}/{DINO_ADAPT_EPOCHS}")
    for first_view, second_view in progress:
        schedule_position = global_step / max(total_steps - 1, 1)
        learning_rate_multiplier = 0.5 * (
            1.0 + math.cos(math.pi * schedule_position)
        )
        teacher_momentum = 1.0 - (1.0 - DINO_ADAPT_TEACHER_MOMENTUM) * 0.5 * (
            1.0 + math.cos(math.pi * schedule_position)
        )
        for parameter_group in optimizer.param_groups:
            parameter_group["lr"] = (
                parameter_group["base_lr"] * learning_rate_multiplier
            )

        images = torch.cat([first_view, second_view], dim=0).to(
            DEVICE, non_blocking=True
        )
        optimizer.zero_grad(set_to_none=True)
        with torch.autocast(
            device_type=DEVICE.type,
            dtype=torch.float16,
            enabled=DEVICE.type == "cuda",

```

```

):
    student_logits = student_head(student_backbone(images))
    with torch.no_grad():
        teacher_logits = teacher_head(teacher_backbone(images))
        loss = dino_cross_view_loss(student_logits, teacher_logits, center)

    if not torch.isfinite(loss):
        raise FloatingPointError(f"Non-finite DINO loss at step {global_step}: {loss}")
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(
        trainable_backbone_parameters + list(student_head.parameters()),
        max_norm=3.0,
    )
    scaler.step(optimizer)
    scaler.update()

    ema_update(teacher_backbone, student_backbone, teacher_momentum)
    ema_update(teacher_head, student_head, teacher_momentum)
    with torch.no_grad():
        batch_center = teacher_logits.float().mean(dim=0, keepdim=True)
        center.mul_(DINO_CENTER_MOMENTUM).add_(
            batch_center, alpha=1.0 - DINO_CENTER_MOMENTUM
        )

    current_loss = loss.detach().item()
    running_loss += current_loss
    global_step += 1
    progress.set_postfix(
        loss=f"{current_loss:.4f}",
        momentum=f"{teacher_momentum:.5f}",
    )

epoch_loss = running_loss / max(len(loader), 1)
history.append({"epoch": epoch + 1, "loss": epoch_loss})
print(f"Epoch {epoch + 1}: mean DINO loss={epoch_loss:.5f}")

checkpoint = {
    "state_dict": {
        key: value.detach().cpu()
        for key, value in teacher_backbone.state_dict().items()
    },
    "metadata": {
        "signature": DINO_DOMAIN_SIGNATURE,
        "config": DINO_DOMAIN_CONFIG,
        "adaptation_split": "train_patients_only_unlabelled",
        "n_adaptation_images": len(dino_adaptation_pool),
        "n_adaptation_patients": int(dino_adaptation_pool["patient_id"].nunique()),
        "history": history,
    },
}
temporary_path = DINO_DOMAIN_CHECKPOINT.with_suffix(".tmp.pt")
torch.save(checkpoint, temporary_path)
temporary_path.replace(DINO_DOMAIN_CHECKPOINT)
print("Saved domain-adapted DINO checkpoint:", DINO_DOMAIN_CHECKPOINT)

del student_backbone, teacher_backbone, student_head, teacher_head
if DEVICE.type == "cuda":
    torch.cuda.empty_cache()
return checkpoint

dino_domain_checkpoint = train_or_load_domain_dino()
assert dino_domain_checkpoint["metadata"]["signature"] == DINO_DOMAIN_SIGNATURE

# The SHA is intentionally computed only after the checkpoint exists.
DINO_DOMAIN_CHECKPOINT_SHA256 = file_sha256(DINO_DOMAIN_CHECKPOINT)
pd.DataFrame([
    {
        "checkpoint": str(DINO_DOMAIN_CHECKPOINT),
        "signature": DINO_DOMAIN_SIGNATURE,
        "sha256": DINO_DOMAIN_CHECKPOINT_SHA256,
    }
]).to_csv(RESULT_DIR / "dino_checkpoint_provenance.csv", index=False)

display(pd.DataFrame(dino_domain_checkpoint["metadata"].get("history", [])))
print("Checkpoint SHA256:", DINO_DOMAIN_CHECKPOINT_SHA256)

```

Using matching domain-adapted DINO checkpoint: /content/drive/MyDrive/isic\_label\_efficiency\_v1/models/dinov2\_isic\_domain\_adapted

| epoch | loss |  |
|-------|------|-----------------------------------------------------------------------------------|
|-------|------|-----------------------------------------------------------------------------------|

|   |   |          |
|---|---|----------|
| 0 | 1 | 7.050343 |
|---|---|----------|

|   |   |          |
|---|---|----------|
| 1 | 2 | 5.442726 |
|---|---|----------|

Checkpoint SHA256: bf9e00858ff80f1073db6be9129bfc45b03a9a3ede67b7b5857f5b2f56cc52e6

# step 10 - Define each backbone once and verify/migrate feature caches

```
MODEL_SPECS = {
    "resnet50_in": {
        "model_name": "resnet50.a1_in1k",
        "kwargs": {},
        "description": "supervised ImageNet ResNet-50",
        "display_name": "ImageNet ResNet-50",
        "architecture": "cnn",
    },
    "vits_in": {
        "model_name": "vit_small_patch16_224.augreg_in21k_ft_in1k",
        "kwargs": {},
        "description": "supervised ImageNet ViT-S/16",
        "display_name": "Supervised ViT-S/16",
        "architecture": "vit",
    },
    "dinov2_reg": {
        "model_name": DINO_BASE_MODEL_NAME,
        "kwargs": {"img_size": IMAGE_SIZE},
        "description": "self-supervised DINOv2 ViT-S/14 with registers",
        "display_name": "DINOv2 ViT-S/14",
        "architecture": "vit",
    },
    DINO_DOMAIN_MODEL_KEY: {
        "model_name": DINO_BASE_MODEL_NAME,
        "kwargs": {"img_size": IMAGE_SIZE},
        "description": "DINOv2 ViT-S/14 adapted on unlabelled ISIC training images",
        "display_name": "Domain-adapted DINOv2",
        "architecture": "vit",
        "checkpoint": str(DINO_DOMAIN_CHECKPOINT),
        "checkpoint_signature": DINO_DOMAIN_SIGNATURE,
        "checkpoint_sha256": DINO_DOMAIN_CHECKPOINT_SHA256,
    },
}

def build_backbone(model_key):
    spec = MODEL_SPECS[model_key]
    checkpoint_path = spec.get("checkpoint")
    model = timm.create_model(
        spec["model_name"],
        pretrained=checkpoint_path is None,
        num_classes=0,
        **spec["kwargs"],
    )
    if checkpoint_path is not None:
        checkpoint_path = Path(checkpoint_path)
        if not checkpoint_path.is_file():
            raise FileNotFoundError(f"Missing adapted checkpoint: {checkpoint_path}")
        checkpoint = load_checkpoint_safely(checkpoint_path)
        saved_signature = checkpoint.get("metadata", {}).get("signature")
        assert saved_signature == spec["checkpoint_signature"], (
            f"Checkpoint signature mismatch for {model_key}: {saved_signature}"
        )
        assert file_sha256(checkpoint_path) == spec["checkpoint_sha256"]
        model.load_state_dict(checkpoint["state_dict"], strict=True)
    return model

@torch.inference_mode()
def extract_features(model, frame):
    model = model.to(DEVICE).eval()
    loader = DataLoader(
        LesionDataset(frame, eval_transform),
        batch_size=BATCH_SIZE,
        shuffle=False,
        num_workers=NUM_WORKERS,
        pin_memory=PIN_MEMORY,
        persistent_workers=NUM_WORKERS > 0,
    )
    feature_batches = []
```

```

label_batches = []
for images, labels in tqdm(loader, desc="Extracting", leave=False):
    images = images.to(DEVICE, non_blocking=True)
    with torch.autocast(
        device_type=DEVICE.type,
        dtype=torch.float16,
        enabled=DEVICE.type == "cuda",
    ):
        features = model(images)
    assert isinstance(features, torch.Tensor) and features.ndim == 2
    feature_batches.append(features.float().cpu())
    label_batches.append(labels.cpu())
return torch.cat(feature_batches).numpy(), torch.cat(label_batches).numpy()

def feature_cache_signature(frame, model_key, model_spec=None):
    frame_bytes = pd.util.hash_pandas_object(
        frame[["isic_id", "patient_id", "target"]], index=False
    ).values.tobytes()
    config = {
        "dataset": DATASET_HANDLE,
        "model": MODEL_SPECS[model_key] if model_spec is None else model_spec,
        "image_size": IMAGE_SIZE,
        "preprocessing": "resize256-bicubic-centercrop-imagenet-v1",
        "timm": timm.__version__,
        "torch": torch.__version__,
    }
    return hashlib.sha256(
        frame_bytes + json.dumps(config, sort_keys=True).encode()
    ).hexdigest()[:16]

def feature_cache_path(model_key, split_name, frame, model_spec=None):
    signature = feature_cache_signature(frame, model_key, model_spec=model_spec)
    return FEATURE_DIR / f"{model_key}_{split_name}_{signature}.npz"

def read_and_validate_feature_cache(path, frame):
    expected_ids = np.asarray(frame["isic_id"].astype(str), dtype="U")
    expected_patients = np.asarray(frame["patient_id"].astype(str), dtype="U")
    expected_y = frame["target"].to_numpy(dtype=np.float32)
    with np.load(path, allow_pickle=False) as cached:
        X = cached["X"].astype(np.float32)
        y = cached["y"].astype(np.float32)
        ids = cached["isic_id"].astype(str)
        patients = cached["patient_id"].astype(str)
    assert np.array_equal(ids, expected_ids), f"Stale ID cache: {path}"
    assert np.array_equal(patients, expected_patients), f"Stale patient cache: {path}"
    assert np.array_equal(y, expected_y), f"Stale label cache: {path}"
    assert len(X) == len(frame)
    return X, y, ids, patients

def migrate_verified_legacy_domain_caches():
    if not ALLOW_VERIFIED_LEGACY_CACHE_MIGRATION:
        return

    legacy_spec = copy.deepcopy(MODEL_SPECS[DINO_DOMAIN_MODEL_KEY])
    legacy_spec["checkpoint_sha256"] = None
    split_frames = {"probe_pool": probe_pool, "validation": val_df}
    migrations = []
    for split_name, frame in split_frames.items():
        current_path = feature_cache_path(DINO_DOMAIN_MODEL_KEY, split_name, frame)
        legacy_path = feature_cache_path(
            DINO_DOMAIN_MODEL_KEY,
            split_name,
            frame,
            model_spec=legacy_spec,
        )
        if not current_path.exists() and legacy_path.exists():
            migrations.append((split_name, frame, legacy_path, current_path))

    if not migrations:
        return

    print("Found legacy domain cache(s). Running a small numerical audit...")
    backbone = build_backbone(DINO_DOMAIN_MODEL_KEY).to(DEVICE).eval()
    try:
        for split_name, frame, legacy_path, current_path in migrations:
            X_cached, _, _, _ = read_and_validate_feature_cache(legacy_path, frame)
            audit_count = min(8, len(frame))
            audit_positions = np.linspace(

```

```

        0, len(frame) - 1, num=audit_count, dtype=int
    )
    audit_frame = frame.iloc[audit_positions].reset_index(drop=True)
    X_live, _ = extract_features(backbone, audit_frame)

    cached_norm = X_cached[audit_positions]
    cached_norm /= np.linalg.norm(cached_norm, axis=1, keepdims=True) + 1e-8
    live_norm = X_live
    live_norm /= np.linalg.norm(live_norm, axis=1, keepdims=True) + 1e-8
    cosine = np.sum(cached_norm * live_norm, axis=1)
    if float(cosine.min()) < 0.999:
        raise AssertionError(
            f"Legacy {split_name} cache failed audit: min cosine={cosine.min():.6f}"
        )

    temporary = current_path.with_name(current_path.stem + ".migrating.npz")
    shutil.copy2(legacy_path, temporary)
    temporary.replace(current_path)
    print(
        f"Verified and migrated {split_name}: min cosine={cosine.min():.6f}"
    )
finally:
    del backbone
    if DEVICE.type == "cuda":
        torch.cuda.empty_cache()

def cached_features(model_key, split_name, frame, force=False):
    path = feature_cache_path(model_key, split_name, frame)
    expected_y = frame["target"].to_numpy(dtype=np.float32)

    if path.exists() and not force:
        X, y, ids, patients = read_and_validate_feature_cache(path, frame)
        print(f"Loaded cache {model_key}/{split_name}: {X.shape}")
        return X, y, ids, patients

    if not ALLOW_FEATURE_EXTRACTION:
        raise RuntimeError(
            f"Cache miss for {model_key}/{split_name}: {path}. "
            "Stop and verify the signature. Set ALLOW_FEATURE_EXTRACTION=True "
            "only for a deliberate one-time extraction."
        )

    model = build_backbone(model_key)
    X, y = extract_features(model, frame)
    del model
    if DEVICE.type == "cuda":
        torch.cuda.empty_cache()
    assert np.array_equal(y.astype(np.float32), expected_y)

    expected_ids = np.asarray(frame["isic_id"].astype(str), dtype="U")
    expected_patients = np.asarray(frame["patient_id"].astype(str), dtype="U")
    temporary = path.with_name(path.stem + ".tmp.npz")
    np.savez(
        temporary,
        X=X.astype(np.float16),
        y=expected_y,
        isic_id=expected_ids,
        patient_id=expected_patients,
    )
    temporary.replace(path)
    print(f"Saved cache {model_key}/{split_name}: {X.shape}")
    return X.astype(np.float32), expected_y, expected_ids, expected_patients

migrate_verified_legacy_domain_caches()

```

```
# step 11 - load allcached features
```

```

feature_data = {}
for model_key in MODEL_SPECS:
    started = time.perf_counter()
    feature_data[model_key] = {
        "train": cached_features(model_key, "probe_pool", probe_pool),
        "validation": cached_features(model_key, "validation", val_df),
    }
    elapsed_minutes = (time.perf_counter() - started) / 60
    print(f"{model_key} completed in {elapsed_minutes:.1f} minutes")

```

```

Loaded cache resnet50_in/probe_pool: (11231, 2048)
Loaded cache resnet50_in/validation: (36070, 2048)

```

```

resnet50_in completed in 0.0 minutes
Loaded cache vits_in/probe_pool: (11231, 384)
Loaded cache vits_in/validation: (36070, 384)
vits_in completed in 0.0 minutes
Loaded cache dinov2_reg/probe_pool: (11231, 384)
Loaded cache dinov2_reg/validation: (36070, 384)
dinov2_reg completed in 0.0 minutes
Loaded cache dinov2_domain/probe_pool: (11231, 384)
Loaded cache dinov2_domain/validation: (36070, 384)
dinov2_domain completed in 0.0 minutes

```

```
# step 12 - Define metrics and grouped logistic probes
```

```

SELECTION_METRIC = "pauc80" # metric used for inner-CV selection of C

def l2_normalize(X):
    return X / (np.linalg.norm(X, axis=1, keepdims=True) + 1e-8)

def pauc_above_tpr(y_true, y_score, min_tpr=0.80):
    y_true = np.asarray(y_true)
    y_score = np.asarray(y_score)
    if np.unique(y_true).size < 2:
        return np.nan
    max_fpr = 1.0 - min_tpr
    standardized = roc_auc_score(
        np.abs(y_true - 1), -y_score, max_fpr=max_fpr
    )
    return float(
        0.5 * max_fpr**2
        + (max_fpr - 0.5 * max_fpr**2) / 0.5 * (standardized - 0.5)
    )

def all_metrics(y_true, y_score):
    y_true = np.asarray(y_true)
    y_score = np.asarray(y_score)
    # Calculate accuracy using a threshold of 0.5
    accuracy = float((y_true == (y_score > 0.5)).mean())
    return {
        "roc_auc": float(roc_auc_score(y_true, y_score)),
        "pr_auc": float(average_precision_score(y_true, y_score)),
        "pauc80": pauc_above_tpr(y_true, y_score),
        "accuracy": accuracy
    }

def selection_score(y_true, y_score, metric=SELECTION_METRIC):
    """Score used to choose C. Kept separate from all_metrics so the
    selection criterion and the reporting criterion are visibly the same
    quantity."""
    if metric == "pauc80":
        return pauc_above_tpr(y_true, y_score)
    if metric == "roc_auc":
        return float(roc_auc_score(y_true, y_score))
    if metric == "pr_auc":
        return float(average_precision_score(y_true, y_score))
    raise KeyError(metric)

def patient_bootstrap_ci(
    y_true, y_score, patient_ids, metric="pauc80", n_boot=N_FULL_PATIENT_BOOTSTRAPS, seed=SEED
):
    y_true = np.asarray(y_true)
    y_score = np.asarray(y_score)
    patient_ids = np.asarray(patient_ids)
    patients = np.unique(patient_ids)
    indices = {p: np.flatnonzero(patient_ids == p) for p in patients}
    rng = np.random.default_rng(seed)
    values = []
    for _ in range(n_boot):
        sampled_patients = rng.choice(patients, size=len(patients), replace=True)
        sampled_indices = np.concatenate([indices[p] for p in sampled_patients])
        y_sample = y_true[sampled_indices]
        if np.unique(y_sample).size < 2:
            continue
        score_sample = y_score[sampled_indices]
        values.append(all_metrics(y_sample, score_sample)[metric])
    if not values:
        return np.nan, np.nan
    return tuple(float(v) for v in np.percentile(values, [2.5, 97.5]))

def fit_grouped_probe(X, y, patient_ids, seed):

```

```

"""Returns (classifier, C, mode, inner_cv_score).

inner_cv_score is np.nan whenever C was not chosen by inner CV.
"""
X = l2_normalize(np.asarray(X, dtype=np.float32))
y = np.asarray(y, dtype=np.int64)
patient_ids = np.asarray(patient_ids)

if np.unique(y).size < 2:
    return None, np.nan, "constant_one_class", np.nan

positive_patients = np.unique(patient_ids[y == 1])

def fit_at_C(C):
    classifier = LogisticRegression(
        C=C,
        max_iter=3000,
        class_weight="balanced",
        solver="lbfgs",
        random_state=seed,
    )
    classifier.fit(X, y)
    return classifier

if len(positive_patients) < 3:
    return fit_at_C(DEFAULT_C), DEFAULT_C, "prespecified_default", np.nan

n_splits = min(3, len(positive_patients))
cv = StratifiedGroupKFold(n_splits=n_splits, shuffle=True, random_state=seed)
try:
    folds = list(cv.split(X, y, groups=patient_ids))
except ValueError:
    return fit_at_C(DEFAULT_C), DEFAULT_C, "prespecified_default", np.nan

if any(
    np.unique(y[train_idx]).size < 2 or np.unique(y[val_idx]).size < 2
    for train_idx, val_idx in folds
):
    return fit_at_C(DEFAULT_C), DEFAULT_C, "prespecified_default", np.nan

best_C = DEFAULT_C
best_score = -np.inf
for C in C_GRID:
    out_of_fold = np.full(len(y), np.nan, dtype=np.float64)
    for train_idx, val_idx in folds:
        model = LogisticRegression(
            C=C,
            max_iter=3000,
            class_weight="balanced",
            solver="lbfgs",
            random_state=seed,
        )
        model.fit(X[train_idx], y[train_idx])
        out_of_fold[val_idx] = model.predict_proba(X[val_idx])[:, 1]
    score = selection_score(y, out_of_fold)
    if not np.isfinite(score):
        continue
    if score > best_score:
        best_C = C
        best_score = score

if not np.isfinite(best_score):
    return fit_at_C(DEFAULT_C), DEFAULT_C, "prespecified_default", np.nan

return fit_at_C(best_C), best_C, "grouped_inner_cv", float(best_score)

```

# step 13 - Run every label budget at 20 seed

```

# Generate each patient budget once so every backbone receives identical patients.
budget_selections = {}
for fraction in LABEL_FRACTIONS:
    for budget_seed in BUDGET_SEEDS:
        chosen_patients, annotation_report = select_patient_budget(
            train_pool, fraction, budget_seed
        )
        budget_selections[(fraction, budget_seed)] = {
            "patients": chosen_patients,
            "report": annotation_report,
        }

```

```

full_patient_sets = {
    tuple(budget_selections[(1.0, seed)][("patients")])
    for seed in BUDGET_SEEDS
}
assert len(full_patient_sets) == 1

rows = []
validation_predictions = {}
full_validation_predictions = {} # predeclared reference seed only

for model_key in MODEL_SPECS:
    X_train, y_train, _, train_patient_ids = feature_data[model_key]["train"]
    X_val, y_val, val_ids, val_patient_ids = feature_data[model_key]["validation"]
    X_val_normalized = l2_normalize(X_val)

    for fraction in LABEL_FRACTIONS:
        for budget_seed in BUDGET_SEEDS:
            selection = budget_selections[(fraction, budget_seed)]
            chosen_patients = selection["patients"]
            annotation_report = selection["report"]
            mask = np.isin(train_patient_ids, chosen_patients)
            X_budget = X_train[mask]
            y_budget = y_train[mask]
            patient_budget_ids = train_patient_ids[mask]

            classifier, selected_C, mode, inner_score = fit_grouped_probe(
                X_budget,
                y_budget,
                patient_budget_ids,
                budget_seed,
            )

            if classifier is None:
                # A constant score has the chance-ranking pAUC under the same metric.
                constant = float(y_budget.mean()) if len(y_budget) else 0.0
                validation_scores = np.full(len(y_val), constant, dtype=np.float64)
            else:
                validation_scores = classifier.predict_proba(X_val_normalized)[: , 1]

            metrics = all_metrics(y_val, validation_scores)
            positive_fit_patients = np.unique(
                patient_budget_ids[y_budget == 1]
            ).size

            rows.append({
                "backbone": model_key,
                "label_fraction": fraction,
                "seed": budget_seed,
                "trainable": classifier is not None,
                "C": selected_C,
                "C_mode": mode,
                "inner_cv_score": inner_score,
                "selection_metric": SELECTION_METRIC,
                "n_fit_images": int(mask.sum()),
                "n_fit_positive_images": int(y_budget.sum()),
                "n_fit_positive_patients": int(positive_fit_patients),
                **annotation_report,
                **metrics,
            })

            prediction = {
                "y": y_val.copy(),
                "score": validation_scores.astype(np.float32),
                "isic_id": val_ids.copy(),
                "patient_id": val_patient_ids.copy(),
                "trainable": classifier is not None,
                "C": selected_C,
                "C_mode": mode,
            }
            validation_predictions[(model_key, fraction, budget_seed)] = prediction

            if fraction == 1.0 and budget_seed == REFERENCE_FULL_SEED:
                full_validation_predictions[model_key] = prediction

        print("Completed", model_key)

results = pd.DataFrame(rows)
results.to_csv(RESULT_DIR / "validation_probe_runs.csv", index=False)

group_keys = ["backbone", "label_fraction"]
run_counts = (
    results.groupby(group_keys, as_index=False)

```

```

        .agg(
            n_runs=("seed", "size"),
            n_trainable=("trainable", "sum"),
            trainable_rate=("trainable", "mean"),
        )
    )

failed = (
    results[~results["trainable"]]
    .groupby(group_keys)["seed"]
    .apply(lambda values: ", ".join(str(int(v)) for v in sorted(values)))
    .rename("failed_seeds")
    .reset_index()
)

annotation_summary = (
    results.groupby(group_keys, as_index=False)
    .agg(
        mean_labelled_patients=("n_labelled_patients", "mean"),
        mean_labelled_images=("n_labelled_images", "mean"),
        mean_positive_patients=("n_labelled_positive_patients", "mean"),
    )
)

trainable_only = results[results["trainable"]].copy()
conditional_summary = (
    trainable_only.groupby(group_keys, as_index=False)
    .agg(
        pauc80_conditional=("pauc80", "mean"),
        pauc80_conditional_sd=("pauc80", "std"),
        pr_auc_conditional=("pr_auc", "mean"),
        roc_auc_conditional=("roc_auc", "mean"),
        accuracy_conditional=("accuracy", "mean"), # Added accuracy conditional
    )
)

all_run_summary = (
    results.groupby(group_keys, as_index=False)
    .agg(
        pauc80_all_runs=("pauc80", "mean"),
        pauc80_seed_sd=("pauc80", "std"),
        pr_auc_all_runs=("pr_auc", "mean"),
        pr_auc_seed_sd=("pr_auc", "std"),
        roc_auc_all_runs=("roc_auc", "mean"),
        roc_auc_seed_sd=("roc_auc", "std"),
        accuracy_all_runs=("accuracy", "mean"), # Added accuracy all runs
        accuracy_seed_sd=("accuracy", "std"), # Added accuracy std
    )
)

summary = (
    run_counts
    .merge(annotation_summary, on=group_keys, how="left")
    .merge(conditional_summary, on=group_keys, how="left")
    .merge(all_run_summary, on=group_keys, how="left")
    .merge(failed, on=group_keys, how="left")
)

summary["failed_seeds"] = summary["failed_seeds"].fillna("")
summary.to_csv(RESULT_DIR / "validation_probe_summary.csv", index=False)

expected_runs = len(MODEL_SPECS) * len(LABEL_FRACTIONS) * len(BUDGET_SEEDS)
assert len(results) == expected_runs
assert results.groupby(group_keys).size().eq(len(BUDGET_SEEDS)).all()
assert results["pauc80"].notna().all()

display(summary.round(4))
n_failed = int(~results["trainable"].sum())
print(f"Unfittable attempted runs retained at chance: {n_failed} of {len(results)}")

```

Completed resnet50\_in  
 Completed vits\_in  
 Completed dinov2\_reg  
 Completed dinov2\_domain

|    | backbone      | label_fraction | n_runs | n_trainable | trainable_rate | mean_labelled_patients | mean_labelled_images | mean_pc |
|----|---------------|----------------|--------|-------------|----------------|------------------------|----------------------|---------|
| 0  | dinov2_domain | 0.05           | 20     | 19          | 0.95           | 19.0                   | 6930.75              |         |
| 1  | dinov2_domain | 0.10           | 20     | 20          | 1.00           | 38.0                   | 13212.50             |         |
| 2  | dinov2_domain | 0.25           | 20     | 20          | 1.00           | 95.0                   | 33047.15             |         |
| 3  | dinov2_domain | 1.00           | 20     | 20          | 1.00           | 381.0                  | 134874.00            |         |
| 4  | dinov2_reg    | 0.05           | 20     | 19          | 0.95           | 19.0                   | 6930.75              |         |
| 5  | dinov2_reg    | 0.10           | 20     | 20          | 1.00           | 38.0                   | 13212.50             |         |
| 6  | dinov2_reg    | 0.25           | 20     | 20          | 1.00           | 95.0                   | 33047.15             |         |
| 7  | dinov2_reg    | 1.00           | 20     | 20          | 1.00           | 381.0                  | 134874.00            |         |
| 8  | resnet50_in   | 0.05           | 20     | 19          | 0.95           | 19.0                   | 6930.75              |         |
| 9  | resnet50_in   | 0.10           | 20     | 20          | 1.00           | 38.0                   | 13212.50             |         |
| 10 | resnet50_in   | 0.25           | 20     | 20          | 1.00           | 95.0                   | 33047.15             |         |
| 11 | resnet50_in   | 1.00           | 20     | 20          | 1.00           | 381.0                  | 134874.00            |         |
| 12 | vits_in       | 0.05           | 20     | 19          | 0.95           | 19.0                   | 6930.75              |         |
| 13 | vits_in       | 0.10           | 20     | 20          | 1.00           | 38.0                   | 13212.50             |         |
| 14 | vits_in       | 0.25           | 20     | 20          | 1.00           | 95.0                   | 33047.15             |         |
| 15 | vits_in       | 1.00           | 20     | 20          | 1.00           | 381.0                  | 134874.00            |         |

16 rows × 22 columns

Unfittable attempted runs retained at chance: 4 of 320

# step 14 - Create the primary line plot

```
fig, ax = plt.subplots(figsize=(8, 5))

for model_key, group in summary.groupby("backbone"):
    group = group.sort_values("label_fraction")
    x = group["label_fraction"].to_numpy() * 100
    y = group["pauc80_all_runs"].to_numpy()
    yerr = group["pauc80_seed_sd"].to_numpy()
    n_trainable = group["n_trainable"].to_numpy()
    n_runs = group["n_runs"].to_numpy()

    line, = ax.plot(x, y, marker="o", label=MODEL_SPECS[model_key]["display_name"])
    ax.errorbar(
        x,
        y,
        yerr=yerr,
        fmt="none",
        capsize=3,
        ecolor=line.get_color(),
    )

    incomplete = n_trainable < n_runs
    if incomplete.any():
        ax.plot(
            x[incomplete],
            y[incomplete],
            linestyle="none",
            marker="o",
            markersize=9,
            markerfacecolor="white",
            markeredgecolor=line.get_color(),
        )
        for xi, yi, fitted, attempted in zip(
            x[incomplete], y[incomplete], n_trainable[incomplete], n_runs[incomplete]
        ):
            ax.annotate(
                f"fittable={int(fitted)}/{int(attempted)}",
                (xi, yi),
                textcoords="offset points",
                xytext=(6, -12),
                fontsize=8,
                color=line.get_color(),
            )
```

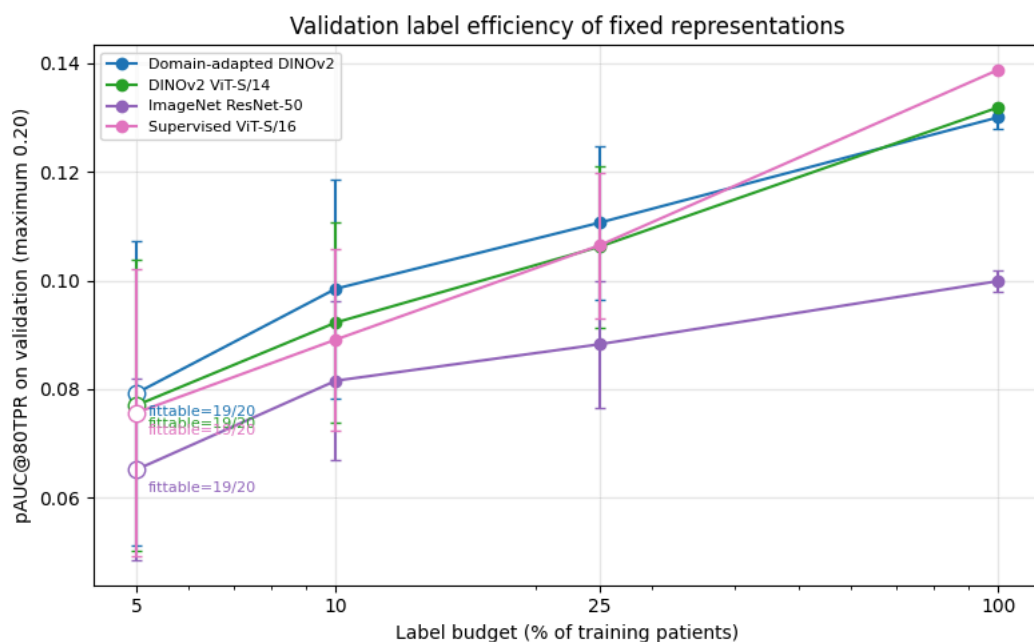
```

    )

    ax.set_xscale("log")
    ax.set_xticks([5, 10, 25, 100])
    ax.get_xaxis().set_major_formatter(plt.ScalarFormatter())
    ax.set_xlabel("Label budget (% of training patients)")
    ax.set_ylabel("pAUC@80TPR on validation (maximum 0.20)")
    ax.set_title("Validation label efficiency of fixed representations")
    ax.grid(alpha=0.3)
    ax.legend(fontsize=8)
    fig.text(
        0.5,
        -0.03,
        "Means and SDs use all 20 attempted seeds. No-positive budgets receive the "
        "constant-predictor chance score. At 100%, seeds vary inner-CV selection only.",
        ha="center",
        fontsize=8,
    )
)
fig.tight_layout()

figure_path = RESULT_DIR / "validation_label_efficiency.png"
fig.savefig(figure_path, dpi=200, bbox_inches="tight")
plt.show()
print("Saved:", figure_path)

```



Means and SDs use all 20 attempted seeds. No-positive budgets receive the constant-predictor chance score. At 100%, seeds vary inner-CV selection only.

Saved: /content/drive/MyDrive/isic\_label\_efficiency\_v1/results\_v10\_final\_a/validation\_label\_efficiency.png

# step 15 - Create the matching Partial Area Under the ROC Curve (pAUC) heatmap

```

budget_order = [0.05, 0.10, 0.25, 1.00]
model_order = [DINO_DOMAIN_MODEL_KEY, "dinov2_reg", "vits_in", "resnet50_in"]
model_labels = {key: MODEL_SPECS[key]["display_name"] for key in model_order}

means = (
    summary.pivot(index="label_fraction", columns="backbone", values="pauc80_all_runs")
    .reindex(index=budget_order, columns=model_order)
)
sds = (
    summary.pivot(index="label_fraction", columns="backbone", values="pauc80_seed_sd")
    .reindex(index=budget_order, columns=model_order)
)
n_trainable = (
    summary.pivot(index="label_fraction", columns="backbone", values="n_trainable")
    .reindex(index=budget_order, columns=model_order)
)
n_runs = (
    summary.pivot(index="label_fraction", columns="backbone", values="n_runs")
    .reindex(index=budget_order, columns=model_order)
)

annotations = means.copy().astype(object)
for budget in budget_order:
    for model in model_order:

```

```

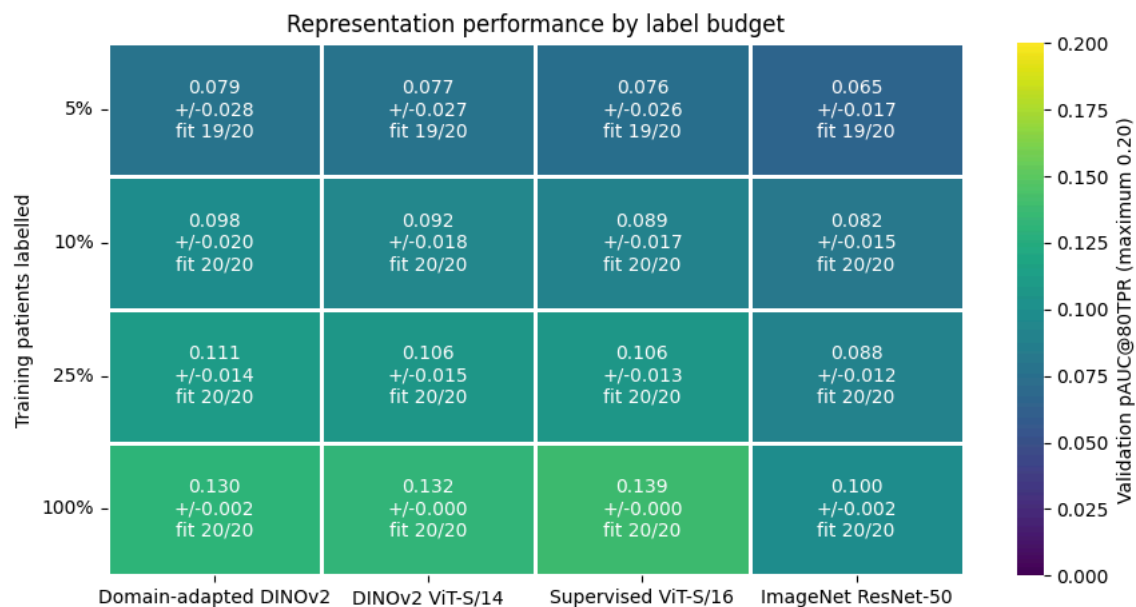
mean = means.loc[budget, model]
sd = sds.loc[budget, model]
fitted = int(n_trainable.loc[budget, model])
attempted = int(n_runs.loc[budget, model])
annotations.loc[budget, model] = (
    f"{mean:.3f}\n+/-{sd:.3f}\nfit {fitted}/{attempted}"
)

display_means = means.copy()
display_annotations = annotations.copy()
display_means.index = ["5%", "10%", "25%", "100%"]
display_means.columns = [model_labels[model] for model in model_order]
display_annotations.index = display_means.index
display_annotations.columns = display_means.columns

fig, ax = plt.subplots(figsize=(9, 5))
sns.heatmap(
    display_means,
    annot=display_annotations,
    fmt="",
    cmap="viridis",
    vmin=0,
    vmax=0.20,
    linewidths=0.8,
    linecolor="white",
    cbar_kws={"label": "Validation pAUC@80TPR (maximum 0.20)"},
    ax=ax,
)
ax.set_title("Representation performance by label budget")
ax.set_xlabel("")
ax.set_ylabel("Training patients labelled")
ax.tick_params(axis="x", rotation=0)
ax.tick_params(axis="y", rotation=0)
fig.text(
    0.5,
    0.01,
    "Cells show all-attempted-run mean, seed SD, and fittable/attempted count. "
    "Chance pAUC is approximately 0.020.",
    ha="center",
    fontsize=8,
)
fig.tight_layout(rect=[0, 0.05, 1, 1])

heatmap_path = RESULT_DIR / "validation_pauc80_heatmap.png"
fig.savefig(heatmap_path, dpi=200, bbox_inches="tight")
plt.show()
print("Saved:", heatmap_path)

```



Cells show all-attempted-run mean, seed SD, and fittable/attempted count. Chance pAUC is approximately 0.020.

Saved: /content/drive/MyDrive/isic\_label\_efficiency\_v1/results\_v10\_final\_a/validation\_pauc80\_heatmap.png

# step 16 - Full budget patient bootstrap for one predeclared seed

```
assert set(full_validation_predictions) == set(MODEL_SPECS)
```

```
full_ci_rows = []
```

```

for model_key, prediction in full_validation_predictions.items():
    metrics = all_metrics(prediction["y"], prediction["score"])
    low, high = patient_bootstrap_ci(
        prediction["y"],
        prediction["score"],
        prediction["patient_id"],
        metric="pauc80",
        n_boot=N_FULL_PATIENT_BOOTSTRAPS,
        seed=SEED,
    )
    full_ci_rows.append({
        "backbone": model_key,
        "reference_seed": REFERENCE_FULL_SEED,
        "n_bootstrap_replicates": N_FULL_PATIENT_BOOTSTRAPS,
        **metrics,
        "pauc80_ci_low": low,
        "pauc80_ci_high": high,
    })

full_validation_ci = pd.DataFrame(full_ci_rows)
full_validation_ci.to_csv(
    RESULT_DIR / "full_budget_validation_patient_bootstrap.csv",
    index=False,
)
display(full_validation_ci.round(4))

def paired_patient_bootstrap_difference(
    y_true,
    score_a,
    score_b,
    patient_ids,
    n_boot=N_FULL_PATIENT_BOOTSTRAPS,
    seed=SEED,
):
    y_true = np.asarray(y_true)
    score_a = np.asarray(score_a)
    score_b = np.asarray(score_b)
    patient_ids = np.asarray(patient_ids)
    patients = np.unique(patient_ids)
    indices = {patient: np.flatnonzero(patient_ids == patient) for patient in patients}
    rng = np.random.default_rng(seed)
    differences = []

    for _ in range(n_boot):
        sampled_patients = rng.choice(patients, size=len(patients), replace=True)
        sampled_indices = np.concatenate([indices[p] for p in sampled_patients])
        y_sample = y_true[sampled_indices]
        if np.unique(y_sample).size < 2:
            continue
        differences.append(
            pauc_above_tpr(y_sample, score_a[sampled_indices])
            - pauc_above_tpr(y_sample, score_b[sampled_indices])
        )

    point = pauc_above_tpr(y_true, score_a) - pauc_above_tpr(y_true, score_b)
    if not differences:
        return float(point), np.nan, np.nan, 0
    low, high = np.percentile(differences, [2.5, 97.5])
    return float(point), float(low), float(high), len(differences)

dino = full_validation_predictions["dinov2_reg"]
supervised_vit = full_validation_predictions["vits_in"]
assert np.array_equal(dino["isic_id"], supervised_vit["isic_id"])

difference, difference_low, difference_high, difference_replicates = (
    paired_patient_bootstrap_difference(
        supervised_vit["y"],
        supervised_vit["score"],
        dino["score"],
        supervised_vit["patient_id"],
    )
)
print(
    f"Supervised ViT - DINOv2 pAUC difference at seed {REFERENCE_FULL_SEED}: "
    f"{difference:+.4f} (95% patient-bootstrap interval "
    f"{difference_low:+.4f} to {difference_high:+.4f}); "
    f"{difference_replicates} valid replicates"
)

```

|   | backbone      | reference_seed | n_bootstrap_replicates | roc_auc | pr_auc | pauc80 | accuracy | pauc80_ci_low | pauc80_ci_high |
|---|---------------|----------------|------------------------|---------|--------|--------|----------|---------------|----------------|
| 0 | resnet50_in   | 0              | 500                    | 0.8470  | 0.0135 | 0.1046 | 0.7907   | 0.0825        | 0.1395         |
| 1 | vits_in       | 0              | 500                    | 0.9103  | 0.0645 | 0.1388 | 0.8714   | 0.1197        | 0.1627         |
| 2 | dinov2_reg    | 0              | 500                    | 0.9025  | 0.0279 | 0.1319 | 0.8719   | 0.0969        | 0.1612         |
| 3 | dinov2_domain | 0              | 500                    | 0.9011  | 0.0310 | 0.1296 | 0.8788   | 0.0936        | 0.1604         |

Supervised ViT - DINOv2 pAUC difference at seed 0: +0.0069 (95% patient-bootstrap interval -0.0085 to +0.0306; 500 valid rep

```
# step 17 - Descriptive paired differences and three primary intervals
```

```
def paired_budget_bootstrap_difference(
    y_true,
    patient_ids,
    scores_a,
    scores_b,
    n_boot=N_PRIMARY_BUDGET_BOOTSTRAPS,
    seed=PRIMARY_BOOTSTRAP_SEED,
):
    """Paired bootstrap over validation patients and all attempted budget seeds."""
    y_true = np.asarray(y_true)
    patient_ids = np.asarray(patient_ids)
    scores_a = np.stack([np.asarray(score, dtype=np.float64) for score in scores_a])
    scores_b = np.stack([np.asarray(score, dtype=np.float64) for score in scores_b])
    assert scores_a.shape == scores_b.shape
    assert scores_a.shape[0] == len(BUDGET_SEEDS)

    patients = np.unique(patient_ids)
    indices = {patient: np.flatnonzero(patient_ids == patient) for patient in patients}
    rng = np.random.default_rng(seed)
    observed = [
        pauc_above_tpr(y_true, score_a) - pauc_above_tpr(y_true, score_b)
        for score_a, score_b in zip(scores_a, scores_b)
    ]
    point = float(np.mean(observed))

    replicates = []
    n_seeds = scores_a.shape[0]
    for _ in tqdm(range(n_boot), leave=False, desc="Bootstrap replicates"):
        sampled_patients = rng.choice(patients, size=len(patients), replace=True)
        sampled_indices = np.concatenate([indices[p] for p in sampled_patients])
        y_sample = y_true[sampled_indices]
        if np.unique(y_sample).size < 2:
            continue
        sampled_seed_indices = rng.integers(0, n_seeds, size=n_seeds)
        per_seed = [
            pauc_above_tpr(
                y_sample,
                scores_a[seed_index, sampled_indices],
            )
            - pauc_above_tpr(
                y_sample,
                scores_b[seed_index, sampled_indices],
            )
            for seed_index in sampled_seed_indices
        ]
        if np.all(np.isfinite(per_seed)):
            replicates.append(float(np.mean(per_seed)))

    if not replicates:
        return point, np.nan, np.nan, 0
    low, high = np.percentile(replicates, [2.5, 97.5])
    return point, float(low), float(high), len(replicates)

reference_run = validation_predictions[(list(MODEL_SPECS)[0], 1.0, REFERENCE_FULL_SEED)]
y_val_shared = np.asarray(reference_run["y"])
patient_ids_shared = np.asarray(reference_run["patient_id"])
isic_ids_shared = np.asarray(reference_run["isic_id"])

for key, run in validation_predictions.items():
    assert np.array_equal(np.asarray(run["isic_id"]), isic_ids_shared), key
    assert np.array_equal(np.asarray(run["y"]), y_val_shared), key
    assert np.array_equal(np.asarray(run["patient_id"]), patient_ids_shared), key

PAIRED_MODEL_COMPARISONS = [
    ("dinov2_domain", "vits_in"),
    ("dinov2_domain", "dinov2_reg"),
    ("dinov2_domain", "resnet50_in"),
```

```

    ("vits_in", "dinov2_reg"),
]

PRIMARY_BOOTSTRAP_COMPARISONS = {
    ("dinov2_domain", "dinov2_reg", 0.10),
    ("dinov2_domain", "dinov2_reg", 0.25),
    ("dinov2_domain", "vits_in", 0.10),
    ("dinov2_domain", "vits_in", 0.25),
    ("dinov2_domain", "vits_in", 1.00),
    ("vits_in", "dinov2_reg", 1.00),
}

paired_rows = []
for model_a, model_b in PAIRED_MODEL_COMPARISONS:
    for fraction in LABEL_FRACTIONS:
        a = results.loc[
            (results["backbone"] == model_a)
            & (results["label_fraction"] == fraction),
            ["seed", "pauc80", "trainable"],
        ].rename(columns={"pauc80": "pauc80_a", "trainable": "trainable_a"})
        b = results.loc[
            (results["backbone"] == model_b)
            & (results["label_fraction"] == fraction),
            ["seed", "pauc80", "trainable"],
        ].rename(columns={"pauc80": "pauc80_b", "trainable": "trainable_b"})
        paired = a.merge(b, on="seed", validate="one_to_one")
        assert len(paired) == len(BUDGET_SEEDS)

        seed_differences = paired["pauc80_a"] - paired["pauc80_b"]
        point = float(seed_differences.mean())
        low = high = np.nan
        n_rep = 0
        bootstrap_scope = "none_descriptive_only"

        key = (model_a, model_b, fraction)
        if key in PRIMARY_BOOTSTRAP_COMPARISONS:
            print(f"Primary bootstrap: {model_a} - {model_b} @ {fraction:.0%}")
            point, low, high, n_rep = paired_budget_bootstrap_difference(
                y_val_shared,
                patient_ids_shared,
                [
                    validation_predictions[(model_a, fraction, seed)]["score"]
                    for seed in BUDGET_SEEDS
                ],
                [
                    validation_predictions[(model_b, fraction, seed)]["score"]
                    for seed in BUDGET_SEEDS
                ],
            )
            bootstrap_scope = "validation_patients_and_all_attempted_budget_seeds"

        paired_rows.append({
            "model_a": model_a,
            "model_b": model_b,
            "label_fraction": fraction,
            "n_attempted_seeds": len(paired),
            "n_both_trainable": int((paired["trainable_a"] & paired["trainable_b"]).sum()),
            "pauc80_difference": point,
            "seed_difference_sd": float(seed_differences.std(ddof=1)),
            "ci_low": low,
            "ci_high": high,
            "excludes_zero": bool(
                np.isfinite(low) and np.isfinite(high) and (low > 0 or high < 0)
            ),
            "n_bootstrap_replicates": n_rep,
            "bootstrap_seed": PRIMARY_BOOTSTRAP_SEED if n_rep else np.nan,
            "bootstrap_scope": bootstrap_scope,
        })
    pd.DataFrame(paired_rows).to_csv(
        RESULT_DIR / "paired_backbone_difference_by_budget.csv",
        index=False,
    )

paired_budget_ci = pd.DataFrame(paired_rows)
display(paired_budget_ci.round(4))

plot_frame = paired_budget_ci.dropna(subset=["ci_low", "ci_high"]).copy()
plot_frame["label"] = (
    plot_frame["model_a"]
    + " - "
    + plot_frame["model_b"]

```

```

+ " @ "
+ (plot_frame["label_fraction"] * 100).astype(int).astype(str)
+ "%"
)

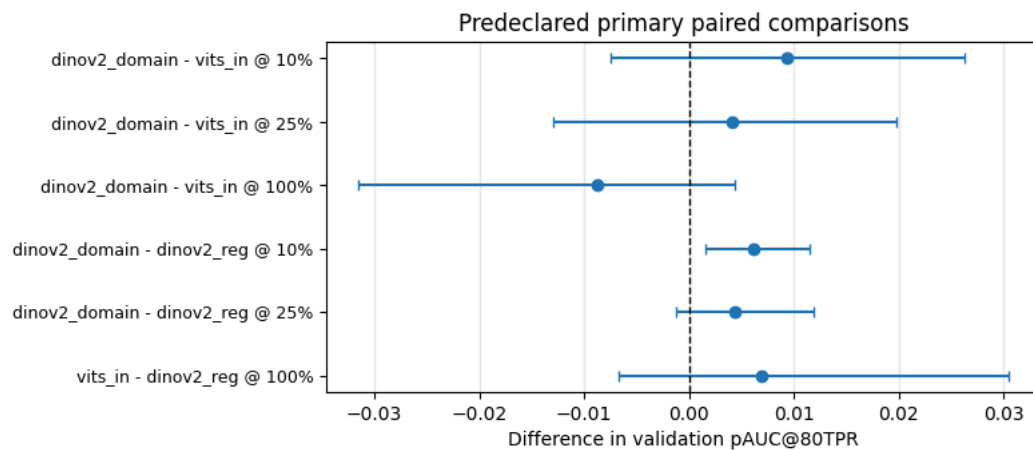
fig, ax = plt.subplots(figsize=(8, 3.8))
positions = np.arange(len(plot_frame))
lower_error = (plot_frame["pauc80_difference"] - plot_frame["ci_low"]).clip(lower=0)
upper_error = (plot_frame["ci_high"] - plot_frame["pauc80_difference"]).clip(lower=0)
ax.errorbar(
    plot_frame["pauc80_difference"],
    positions,
    xerr=[lower_error, upper_error],
    fmt="o",
    capsize=3,
)
ax.axvline(0.0, color="black", linewidth=1, linestyle="--")
ax.set_yticks(positions)
ax.set_yticklabels(plot_frame["label"], fontsize=9)
ax.invert_yaxis()
ax.set_xlabel("Difference in validation pAUC@80TPR")
ax.set_title("Predeclared primary paired comparisons")
ax.grid(axis="x", alpha=0.3)
fig.text(
    0.5,
    0.01,
    f"Intervals use {N_PRIMARY_BUDGET_BOOTSTRAPS} paired bootstrap replicates over "
    "validation patients and all attempted seeds; other CSV rows are descriptive.",
    ha="center",
    fontsize=8,
)
fig.tight_layout(rect=[0, 0.06, 1, 1])

paired_figure_path = RESULT_DIR / "paired_backbone_difference_by_budget.png"
fig.savefig(paired_figure_path, dpi=200, bbox_inches="tight")
plt.show()
print("Saved:", paired_figure_path)

```

Primary bootstrap: dinov2\_domain - vits\_in @ 10%  
 Primary bootstrap: dinov2\_domain - vits\_in @ 25%  
 Primary bootstrap: dinov2\_domain - vits\_in @ 100%  
 Primary bootstrap: dinov2\_domain - dinov2\_reg @ 10%  
 Primary bootstrap: dinov2\_domain - dinov2\_reg @ 25%  
 Primary bootstrap: vits\_in - dinov2\_reg @ 100%

|    | model_a       | model_b     | label_fraction | n_attempted_seeds | n_both_trainable | pauc80_difference | seed_difference_sd |   |
|----|---------------|-------------|----------------|-------------------|------------------|-------------------|--------------------|---|
| 0  | dinov2_domain | vits_in     | 0.05           | 20                | 19               | 0.0036            | 0.0163             |   |
| 1  | dinov2_domain | vits_in     | 0.10           | 20                | 20               | 0.0094            | 0.0138             | - |
| 2  | dinov2_domain | vits_in     | 0.25           | 20                | 20               | 0.0042            | 0.0153             | - |
| 3  | dinov2_domain | vits_in     | 1.00           | 20                | 20               | -0.0088           | 0.0021             | - |
| 4  | dinov2_domain | dinov2_reg  | 0.05           | 20                | 19               | 0.0023            | 0.0084             |   |
| 5  | dinov2_domain | dinov2_reg  | 0.10           | 20                | 20               | 0.0062            | 0.0069             |   |
| 6  | dinov2_domain | dinov2_reg  | 0.25           | 20                | 20               | 0.0044            | 0.0114             | - |
| 7  | dinov2_domain | dinov2_reg  | 1.00           | 20                | 20               | -0.0018           | 0.0021             |   |
| 8  | dinov2_domain | resnet50_in | 0.05           | 20                | 19               | 0.0141            | 0.0197             |   |
| 9  | dinov2_domain | resnet50_in | 0.10           | 20                | 20               | 0.0169            | 0.0145             |   |
| 10 | dinov2_domain | resnet50_in | 0.25           | 20                | 20               | 0.0224            | 0.0153             |   |
| 11 | dinov2_domain | resnet50_in | 1.00           | 20                | 20               | 0.0302            | 0.0031             |   |
| 12 | vits_in       | dinov2_reg  | 0.05           | 20                | 19               | -0.0014           | 0.0138             |   |
| 13 | vits_in       | dinov2_reg  | 0.10           | 20                | 20               | -0.0031           | 0.0135             |   |
| 14 | vits_in       | dinov2_reg  | 0.25           | 20                | 20               | 0.0003            | 0.0164             |   |
| 15 | vits_in       | dinov2_reg  | 1.00           | 20                | 20               | 0.0069            | 0.0000             | - |



Intervals use 2000 paired bootstrap replicates over validation patients and all attempted seeds; other CSV rows are descriptive.

Saved: /content/drive/MyDrive/isic\_label\_efficiency\_v1/results\_v10\_final\_a/paired\_backbone\_difference\_by\_budget.png

# step 18 - Refit the exact 25% probes and select nine shared validation cases

```
GRADCAM_MODEL_ORDER = list(EXPLAIN_MODELS)
GRADCAM_MODEL_LABELS = {
    key: MODEL_SPECS[key]["display_name"] for key in GRADCAM_MODEL_ORDER
}

explanation_probes = {}
configuration_rows = []
selection = budget_selections[(EXPLAIN_FRACTION, EXPLAIN_SEED)]
chosen_explanation_patients = selection["patients"]

for model_key in GRADCAM_MODEL_ORDER:
    X_train, y_train, _, train_patient_ids = feature_data[model_key]["train"]
    mask = np.isin(train_patient_ids, chosen_explanation_patients)
    probe, selected_C, selection_mode, inner_score = fit_grouped_probe(
        X_train[mask],
        y_train[mask],
        train_patient_ids[mask],
        EXPLAIN_SEED,
    )
    assert probe is not None, f"Explanation probe is not trainable for {model_key}"
    assert np.array_equal(probe.classes_, np.array([0, 1]))
```

```

X_val = feature_data[model_key]["validation"][0]
replayed_scores = probe.predict_proba(l2_normalize(X_val))[:, 1]
stored = validation_predictions[(model_key, EXPLAIN_FRACTION, EXPLAIN_SEED)]
np.testing.assert_allclose(
    replayed_scores,
    stored["score"],
    rtol=1e-6,
    atol=1e-7,
)
explanation_probes[model_key] = probe
configuration_rows.append({
    "backbone": model_key,
    "label_fraction": EXPLAIN_FRACTION,
    "seed": EXPLAIN_SEED,
    "C": selected_C,
    "C_mode": selection_mode,
    "inner_cv_score": inner_score,
    "n_labelled_patients": selection["report"]["n_labelled_patients"],
})

explainability_configuration = pd.DataFrame(configuration_rows)
explainability_configuration.to_csv(
    RESULT_DIR / "explainability_configuration.csv", index=False
)
display(explainability_configuration.round(4))

score_frame = None
for model_key in GRADCAM_MODEL_ORDER:
    prediction = validation_predictions[(model_key, EXPLAIN_FRACTION, EXPLAIN_SEED)]
    current = pd.DataFrame({
        "isic_id": prediction["isic_id"].astype(str),
        f"score_{model_key}": prediction["score"],
    })
    score_frame = current if score_frame is None else score_frame.merge(
        current, on="isic_id", validate="one_to_one"
    )

score_columns = [f"score_{model_key}" for model_key in GRADCAM_MODEL_ORDER]
score_frame["reference_score"] = score_frame[score_columns].mean(axis=1)
score_frame["model_disagreement"] = (
    score_frame[score_columns[0]] - score_frame[score_columns[1]]
).abs()

explainability_candidates = val_df[
    ["isic_id", "patient_id", "target", "image_path"]
].copy()
explainability_candidates["isic_id"] = explainability_candidates["isic_id"].astype(str)
explainability_candidates = explainability_candidates.merge(
    score_frame, on="isic_id", how="inner", validate="one_to_one"
)
assert len(explainability_candidates) == len(val_df)

def take_patient_distinct_extreme(
    frame, case_name, sort_column, ascending, excluded_patients=()
):
    eligible = frame[~frame["patient_id"].isin(set(excluded_patients))]
    selected = (
        eligible.sort_values(
            [sort_column, "patient_id", "isic_id"],
            ascending=[ascending, True, True],
        )
        .drop_duplicates("patient_id")
        .head(1)
        .copy()
    )
    assert len(selected) == 1
    selected["case"] = case_name
    return selected

malignant_candidates = explainability_candidates[explainability_candidates["target"] == 1]
benign_candidates = explainability_candidates[explainability_candidates["target"] == 0]
assert len(malignant_candidates) > 0 and len(benign_candidates) > 0

extreme_parts = []
used_extreme_patients = set()
for frame, case_name, sort_column, ascending in [
    (malignant_candidates, "High-ranked malignant", "reference_score", False),
    (malignant_candidates, "Low-ranked malignant", "reference_score", True),
    (benign_candidates, "Largest model disagreement", "model_disagreement", False),
]:

```

```

selected = take_patient_distinct_extreme(
    frame,
    case_name,
    sort_column,
    ascending,
    excluded_patients=used_extreme_patients,
)
used_extreme_patients.update(selected["patient_id"])
extreme_parts.append(selected)

extreme_cases_df = pd.concat(extreme_parts, ignore_index=True)
extreme_cases_df["case_type"] = "extreme"
assert extreme_cases_df["patient_id"].is_unique

extreme_patients = set(extreme_cases_df["patient_id"])
random_pool = explainability_candidates[
    ~explainability_candidates["patient_id"].isin(extreme_patients)
]

random_parts = []
used_random_patients = set()

for target, label, offset in [
    (0, "Random benign", 0),
    (1, "Random malignant", 1),
]:
    candidates = (
        random_pool[
            (random_pool["target"] == target)
            & (~random_pool["patient_id"].isin(used_random_patients))
        ]
        .sort_values(["patient_id", "isic_id"])
        .drop_duplicates("patient_id")
    )

    assert len(candidates) >= RANDOM_CASES_PER_CLASS, (
        f"Only {len(candidates)} eligible {label.lower()} patients"
    )

    selected = candidates.sample(
        n=RANDOM_CASES_PER_CLASS,
        random_state=SEED + offset,
    ).copy()

    selected["case"] = label
    used_random_patients.update(selected["patient_id"])
    random_parts.append(selected)

random_gradcam_cases_df = pd.concat(random_parts, ignore_index=True)
random_gradcam_cases_df["case_type"] = "random"

selected_gradcam_cases = pd.concat(
    [extreme_cases_df, random_gradcam_cases_df], ignore_index=True
)
assert selected_gradcam_cases["patient_id"].is_unique
assert set(selected_gradcam_cases["isic_id"]).issubset(set(val_df["isic_id"].astype(str)))

selected_gradcam_cases.to_csv(
    RESULT_DIR / "explainability_cases.csv",
    index=False,
)
display(
    selected_gradcam_cases[
        ["case_type", "case", "isic_id", "patient_id", "target", *score_columns]
    ].round(4)
)

```

|   | backbone      | label_fraction             | seed         | C          | C_mode           | inner_cv_score | n_labelled_patients |  |
|---|---------------|----------------------------|--------------|------------|------------------|----------------|---------------------|--|
| 0 | vits_in       | 0.25                       | 0            | 10.0       | grouped_inner_cv | 0.1517         | 95                  |  |
| 1 | dinov2_domain | 0.25                       | 0            | 1.0        | grouped_inner_cv | 0.1328         | 95                  |  |
|   | case_type     | case                       | isic_id      | patient_id | target           | score_vits_in  | score_dinov2_domain |  |
| 0 | extreme       | High-ranked malignant      | ISIC_5757798 | IP_1959239 | 1                | 0.9992         | 0.9855              |  |
| 1 | extreme       | Low-ranked malignant       | ISIC_3964079 | IP_1039004 | 1                | 0.0055         | 0.0223              |  |
| 2 | extreme       | Largest model disagreement | ISIC_5760343 | IP_7977194 | 0                | 0.0432         | 0.9228              |  |
| 3 | random        | Random benign              | ISIC_0145174 | IP_1757336 | 0                | 0.1467         | 0.1675              |  |
| 4 | random        | Random benign              | ISIC_0089779 | IP_3796132 | 0                | 0.0012         | 0.0350              |  |
| 5 | random        | Random benign              | ISIC_0101423 | IP_3815550 | 0                | 0.0366         | 0.0994              |  |
| 6 | random        | Random malignant           | ISIC_1480614 | IP_5683980 | 1                | 0.0235         | 0.2670              |  |
| 7 | random        | Random malignant           | ISIC_4416759 | IP_3987348 | 1                | 0.9729         | 0.8851              |  |
| 8 | random        | Random malignant           | ISIC_7231436 | IP_3232631 | 1                | 0.9383         | 0.7853              |  |

# step 19 - Define Grad-CAM and verify the live feature path

```
import torch.nn.functional as F

class LinearProbeGradCAM:
    """Grad-CAM for a timm backbone followed by an sklearn binary probe."""

    def __init__(self, backbone, probe, target_layer, device=DEVICE):
        self.backbone = backbone.to(device).eval()
        self.device = device
        self.activations = None
        self.gradients = None

        # Gradients are needed with respect to activations and the image, not weights.
        for parameter in self.backbone.parameters():
            parameter.requires_grad_(False)

        self.probe_weight = torch.as_tensor(
            probe.coef_[0], dtype=torch.float32, device=device
        )
        self.probe_bias = torch.as_tensor(
            probe.intercept_[0], dtype=torch.float32, device=device
        )
        self.hook = target_layer.register_forward_hook(self._capture_activations)

    def _capture_activations(self, module, inputs, output):
        if isinstance(output, (tuple, list)):
            output = output[0]
        self.activations = output
        output.register_hook(self._capture_gradients)

    def _capture_gradients(self, self, gradient):
        self.gradients = gradient

    def _to_spatial_grid(self, tensor):
        if tensor.ndim == 4:
            return tensor
        if tensor.ndim != 3:
            raise ValueError(f"Expected BCHW or BNC activations, got {tensor.shape}")

        prefix_tokens = int(getattr(self.backbone, "num_prefix_tokens", 1))
        patch_tokens = tensor[:, prefix_tokens:, :]
        grid_size = getattr(self.backbone.patch_embed, "grid_size", None)
        if grid_size is None:
            side = int(round(patch_tokens.shape[1] ** 0.5))
            grid_height, grid_width = side, side
        elif isinstance(grid_size, int):
            grid_height, grid_width = grid_size, grid_size
        else:
            grid_height, grid_width = map(int, grid_size)

        expected_tokens = grid_height * grid_width
        assert patch_tokens.shape[1] == expected_tokens, (
            f"Expected {expected_tokens} patch tokens after removing "
            f"{prefix_tokens} prefix tokens, found {patch_tokens.shape[1]}"
        )
        return (
```

```

        patch_tokens.reshape(
            patch_tokens.shape[0], grid_height, grid_width, patch_tokens.shape[2]
        )
        .permute(0, 3, 1, 2)
        .contiguous()
    )

def __call__(self, images):
    if images.ndim == 3:
        images = images.unsqueeze(0)
    images = images.to(self.device, dtype=torch.float32).detach()
    images.requires_grad_(True)

    self.backbone.zero_grad(set_to_none=True)
    self.activations = None
    self.gradients = None

    with torch.enable_grad():
        features = self.backbone(images)
        normalized_features = F.normalize(features, p=2, dim=1, eps=1e-8)
        malignant_logits = (
            normalized_features @ self.probe_weight + self.probe_bias
        )
        malignant_logits.sum().backward()

    assert self.activations is not None and self.gradients is not None
    activations = self._to_spatial_grid(self.activations)
    gradients = self._to_spatial_grid(self.gradients)

    channel_weights = gradients.mean(dim=(2, 3), keepdim=True)
    raw_cam = torch.relu((channel_weights * activations).sum(dim=1))
    raw_cam = F.interpolate(
        raw_cam.unsqueeze(1),
        size=images.shape[-2:],
        mode="bilinear",
        align_corners=False,
    ).squeeze(1)

    cam_min = raw_cam.amin(dim=(1, 2), keepdim=True)
    cam_max = raw_cam.amax(dim=(1, 2), keepdim=True)
    cam = (raw_cam - cam_min) / (cam_max - cam_min).clamp_min(1e-8)
    probe_scores = torch.sigmoid(malignant_logits)

    return cam.detach().cpu().numpy(), probe_scores.detach().cpu().numpy()

def close(self):
    self.hook.remove()

def gradcam_target_layer(model_key, backbone):
    if model_key == "resnet50_in":
        return backbone.layer4[-1]
    if MODEL_SPECS[model_key]["architecture"] == "vit":
        # The penultimate block retains a spatial patch-token grid.
        return backbone.blocks[-2].norm1
    raise KeyError(model_key)

gradcam_display_transform = transforms.Compose([
    transforms.Resize(256, interpolation=transforms.InterpolationMode.BICUBIC),
    transforms.CenterCrop(IMAGE_SIZE),
])

def prepare_gradcam_image(image_path):
    with Image.open(image_path) as image:
        image = image.convert("RGB")
        display_image = gradcam_display_transform(image)
        model_input = eval_transform(image)
    return np.asarray(display_image).copy(), model_input

```

```

precision_rows = []

for model_key in GRADCAM_MODEL_ORDER:
    stored_prediction = validation_predictions[
        (model_key, EXPLAIN_FRACTION, EXPLAIN_SEED)
    ]
    cached_lookup = dict(zip(
        stored_prediction["isic_id"].astype(str),
        stored_prediction["score"],
    ))

```

```

backbone = build_backbone(model_key).to(DEVICE).eval()
probe = explanation_probes[model_key]
live_scores = []
cached_scores = []
for row in selected_gradcam_cases.itertuples(index=False):
    _, model_input = prepare_gradcam_image(row.image_path)
    with torch.inference_mode():
        features = backbone(model_input.unsqueeze(0).to(DEVICE))
        live = probe.predict_proba(l2_normalize(features.float().cpu().numpy()))[0, 1]
        live_scores.append(float(live))
        cached_scores.append(float(cached_lookup[str(row.isic_id)]))

del backbone
if DEVICE.type == "cuda":
    torch.cuda.empty_cache()

live_scores = np.asarray(live_scores)
cached_scores = np.asarray(cached_scores)
differences = live_scores - cached_scores
largest_index = int(np.argmax(np.abs(differences)))
precision_rows.append({
    "backbone": model_key,
    "label_fraction": EXPLAIN_FRACTION,
    "seed": EXPLAIN_SEED,
    "max_abs_difference": float(np.abs(differences).max()),
    "mean_abs_difference": float(np.abs(differences).mean()),
    "max_abs_difference_isic_id": str(
        selected_gradcam_cases.iloc[largest_index]["isic_id"]
    ),
})

precision_check = pd.DataFrame(precision_rows)
precision_check.to_csv(RESULT_DIR / "gradcam_precision_path_check.csv", index=False)
display(precision_check.round(6))

```

|   | backbone      | label_fraction | seed | max_abs_difference | mean_abs_difference | max_abs_difference_isic_id |
|---|---------------|----------------|------|--------------------|---------------------|----------------------------|
| 0 | vits_in       | 0.25           | 0    | 0.001047           | 0.000225            | ISIC_0145174               |
| 1 | dinov2_domain | 0.25           | 0    | 0.000783           | 0.000251            | ISIC_0145174               |

# step 20- Generate Grad-CAM maps for both models

```

torch.use_deterministic_algorithms(True, warn_only=False)
if DEVICE.type == "cuda":
    torch.backends.cuda.enable_flash_sdp(False)
    torch.backends.cuda.enable_mem_efficient_sdp(False)
    torch.backends.cuda.enable_math_sdp(True)

case_images = {
    row.isic_id: prepare_gradcam_image(row.image_path)
    for row in selected_gradcam_cases.itertuples(index=False)
}

gradcam_results = {}
for model_key in GRADCAM_MODEL_ORDER:
    print(f"Generating Grad-CAM maps for {model_key}...")
    backbone = build_backbone(model_key).to(DEVICE).eval()
    explainer = LinearProbeGradCAM(
        backbone=backbone,
        probe=explanation_probes[model_key],
        target_layer=gradcam_target_layer(model_key, backbone),
    )
    try:
        for row in selected_gradcam_cases.itertuples(index=False):
            _, model_input = case_images[row.isic_id]
            cams, scores = explainer(model_input)
            gradcam_results[(row.isic_id, model_key)] = {
                "cam": cams[0],
                "score": float(scores[0]),
            }
    finally:
        explainer.close()
        del explainer, backbone
        if DEVICE.type == "cuda":
            torch.cuda.empty_cache()

for case_type_to_plot in ["extreme", "random"]:
    current_cases = selected_gradcam_cases[
        selected_gradcam_cases["case_type"] == case_type_to_plot
    ]

```

```

n_rows = len(current_cases)
n_columns = 1 + len(GRADCAM_MODEL_ORDER)
fig, axes = plt.subplots(
    n_rows,
    n_columns,
    figsize=(4.0 * n_columns, 3.7 * n_rows),
    squeeze=False,
)

for row_index, case in enumerate(current_cases.itertuples(index=False)):
    display_image, _ = case_images[case.isic_id]
    truth = "malignant" if int(case.target) == 1 else "benign"
    axes[row_index, 0].imshow(display_image)
    axes[row_index, 0].set_title(f"{case.case}\nTrue label: {truth}")
    axes[row_index, 0].axis("off")

    for column_index, model_key in enumerate(GRADCAM_MODEL_ORDER, start=1):
        result = gradcam_results[(case.isic_id, model_key)]
        axes[row_index, column_index].imshow(display_image)
        axes[row_index, column_index].imshow(
            result["cam"], cmap="magma", alpha=0.48, vmin=0, vmax=1
        )
        axes[row_index, column_index].set_title(
            f"{GRADCAM_MODEL_LABELS[model_key]}\nprobe score={result['score']:.3f}"
        )
        axes[row_index, column_index].axis("off")

fig.suptitle(
    f"Grad-CAM comparison: {case_type_to_plot} cases "
    f"({EXPLAIN_FRACTION:.0%} labels, seed {EXPLAIN_SEED})",
    fontsize=15,
)

fig.text(
    0.5,
    0.01,
    "Maps are normalized independently per panel; colour magnitude is not "
    "comparable between panels.",
    ha="center",
    fontsize=9,
)

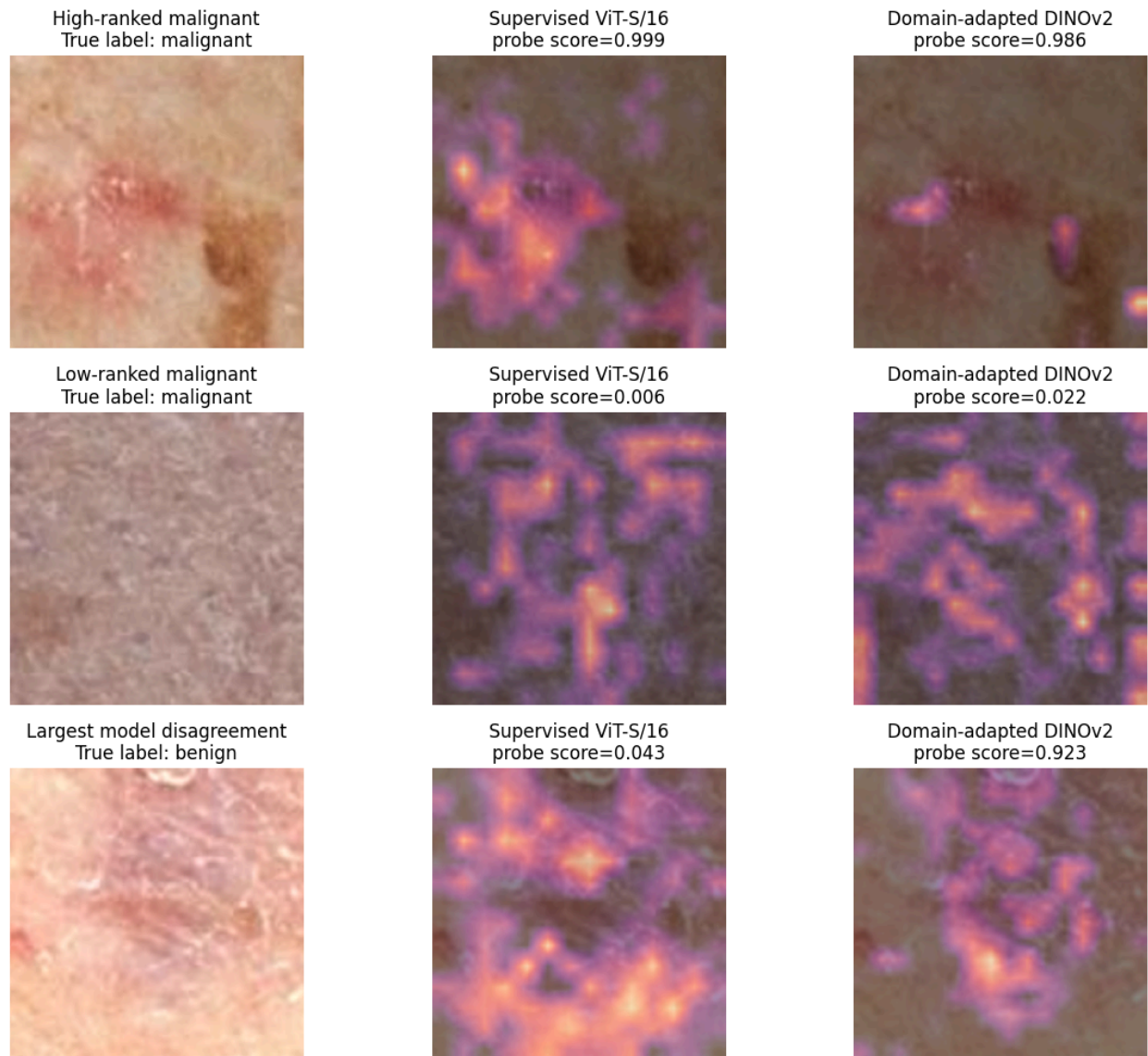
fig.tight_layout(rect=[0, 0.035, 1, 0.965])
path = RESULT_DIR / f"validation_gradcam_comparison_{case_type_to_plot}.png"
fig.savefig(path, dpi=180, bbox_inches="tight")
plt.show()
print("Saved:", path)

```



Generating Grad-CAM maps for vits\_in...  
Generating Grad-CAM maps for dinov2\_domain...

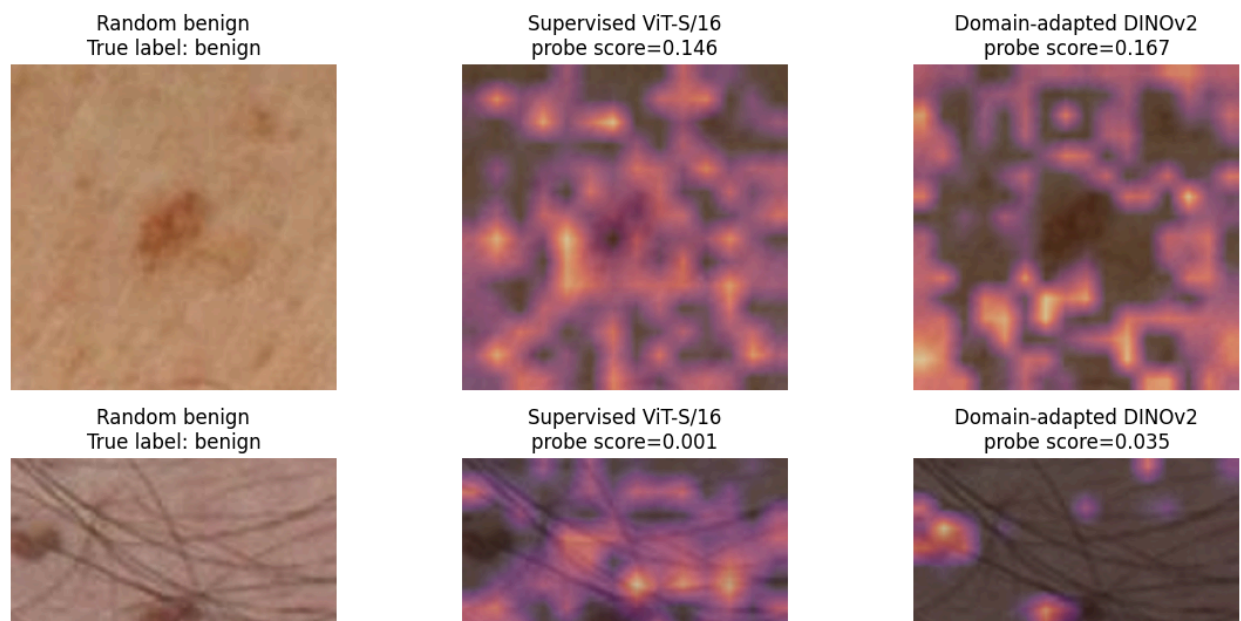
### Grad-CAM comparison: extreme cases (25% labels, seed 0)



Maps are normalized independently per panel; colour magnitude is not comparable between panels.

Saved: /content/drive/MyDrive/isic\_label\_efficiency\_v1/results\_v10\_final\_a/validation\_gradcam\_comparison\_extreme.png

### Grad-CAM comparison: random cases (25% labels, seed 0)

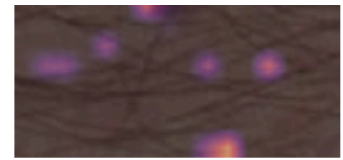




Random benign  
True label: benign



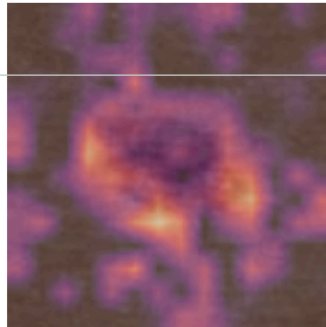
Supervised ViT-S/16  
probe score=0.037



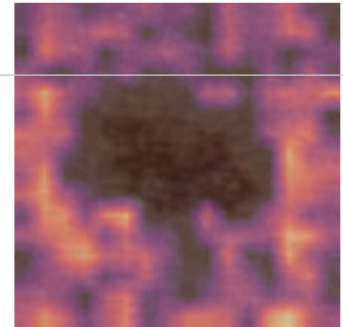
Domain-adapted DINOv2  
probe score=0.099



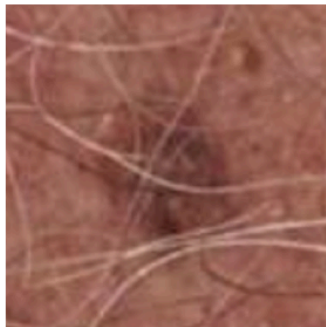
Random malignant  
True label: malignant



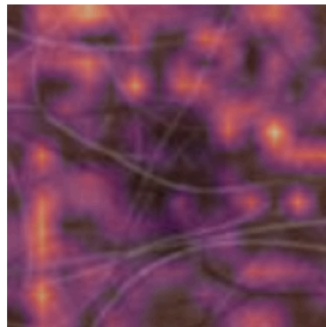
Supervised ViT-S/16  
probe score=0.023



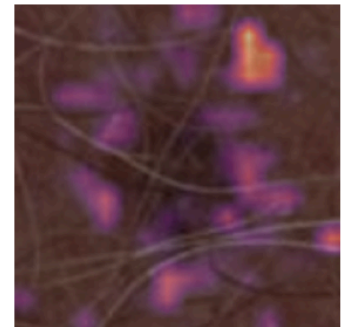
Domain-adapted DINOv2  
probe score=0.267



Random malignant  
True label: malignant



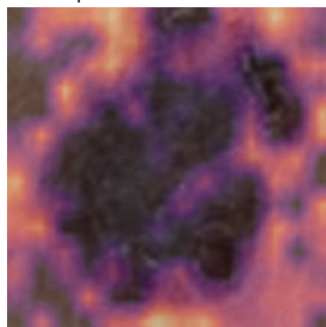
Supervised ViT-S/16  
probe score=0.973



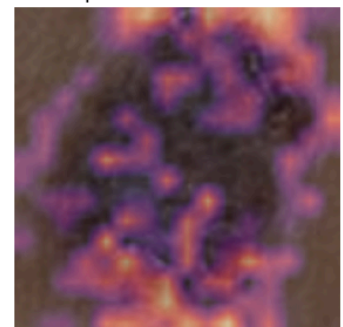
Domain-adapted DINOv2  
probe score=0.885



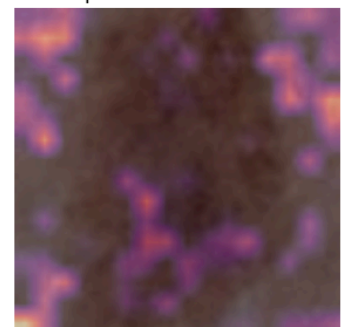
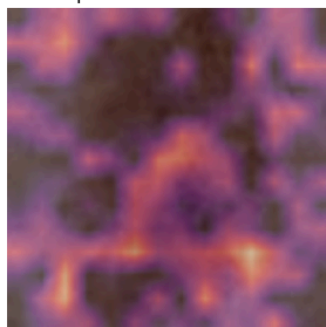
Random malignant  
True label: malignant



Supervised ViT-S/16  
probe score=0.938



Domain-adapted DINOv2  
probe score=0.785



Maps are normalized independently per panel; colour magnitude is not comparable between panels.

Saved: /content/drive/MyDrive/isic\_label\_efficiency\_v1/results\_v10\_final\_a/validation\_gradcam\_comparison\_random.png

```
# step 21 - Generate coarse occlusion-sensitivity maps
```

```
def probe_margin(probe, features):
    normalized = l2_normalize(features.float().cpu().numpy())
    return np.asarray(probe.decision_function(normalized)).reshape(-1)

@torch.inference_mode()
def extract_live_probe_features(backbone, images):
    with torch.autocast(
        device_type=DEVICE.type,
        dtype=torch.float16,
        enabled=DEVICE.type == "cuda",
    ):
        features = backbone(images)
    assert isinstance(features, torch.Tensor) and features.ndim == 2
    return features

@torch.inference_mode()
def occlusion_map(
    backbone,
    probe,
    model_input,
    patch=OCCLUSION_PATCH,
    stride=OCCLUSION_STRIDE,
    batch=OCCLUSION_BATCH,
):
    """Signed decision-margin sensitivity.

    Positive means occluding the region lowered the malignant margin, so the
    region supplied evidence for malignancy.
    """
    backbone.eval()
    channels, height, width = model_input.shape
    assert height >= patch and width >= patch

    ys = list(range(0, height - patch + 1, stride))
    xs = list(range(0, width - patch + 1, stride))
    if ys[-1] != height - patch:
        ys.append(height - patch)
    if xs[-1] != width - patch:
        xs.append(width - patch)
    positions = [(y, x) for y in ys for x in xs]

    base_features = extract_live_probe_features(
        backbone, model_input.unsqueeze(0).to(DEVICE)
    )
    base_margin = float(probe_margin(probe, base_features)[0])
    heat = np.zeros((height, width), dtype=np.float32)
    count = np.zeros((height, width), dtype=np.float32)

    for start in range(0, len(positions), batch):
        chunk = positions[start:start + batch]
        images = model_input.unsqueeze(0).expand(len(chunk), -1, -1, -1).clone()
        for image_index, (y, x) in enumerate(chunk):
            # eval_transform uses ImageNet normalization, so 0 is channel mean.
            images[image_index, :, y:y + patch, x:x + patch] = 0.0

            features = extract_live_probe_features(backbone, images.to(DEVICE))
            scores = probe_margin(probe, features)
            for (y, x), score in zip(chunk, scores):
                heat[y:y + patch, x:x + patch] += base_margin - float(score)
                count[y:y + patch, x:x + patch] += 1.0

    return heat / np.maximum(count, 1.0), base_margin

occlusion_results = {}
for model_key in GRADCAM_MODEL_ORDER:
    print(f"Generating coarse occlusion maps for {model_key}...")
    backbone = build_backbone(model_key).to(DEVICE).eval()
    probe = explanation_probes[model_key]
    try:
        for row in tqdm(
            selected_gradcam_cases.itertuples(index=False),
            total=len(selected_gradcam_cases),
            desc=model_key,
        ):
            _, model_input = case_images[row.isic_id]
            heat, base_margin = occlusion_map(backbone, probe, model_input)
```